

Compact and Statistical NIZK Proofs of Knowledge for Disjunctions from Σ -Protocols

Gennaro Avitabile¹, Luisa Siniscalchi², and Ivan Visconti³

¹ IMDEA Software Institute, Madrid, Spain. avitabilegenn@gmail.com

² Technical University of Denmark (DTU), Kongens Lyngby, Denmark. luisi@dtu.dk

³ Sapienza University of Rome, Rome, Italy. visconti@diag.uniroma1.it

Abstract. The classical results of Cramer et al. [Crypto'94] showed how to compose n Σ -protocols with statistical HVZK to obtain an efficient proof of knowledge of their disjunction maintaining statistical HVZK without adding hardness assumptions. The Fiat-Shamir (FS) transform applied to their construction produces a statistical NIZK proof⁴ of knowledge in the random oracle (RO) model, but, unfortunately, the proof size in their case is linear in n .

Recently, there has been increasing interest in solving the major open problem of obtaining statistical NIZK proofs of knowledge for disjunctions starting from Σ -protocols, with improved communication and minimizing hardness assumptions. The current best results are due 1) to Goel et al. [Eurocrypt '22], which, unfortunately, require Dlog-based assumptions, and 2) to Boudgoust and Simkin [TCC '24] which, unfortunately, obtain computational ZK only. In this work, we solve the above open problem showing, for a large class of Σ -protocols, how to obtain a non-interactive compact statistical NIZK proof of knowledge without adding hardness assumptions, therefore only relying on random oracles. More precisely, the communication complexity of our construction is $O(\lambda^2 \log n) + \text{CC}(\Sigma)$ where λ is the security parameter and $\text{CC}(\Sigma)$ is the communication of a single run of the underlying Σ -protocol, and this is obtained via calls to a random oracle and to a prover executing a stand-alone instance of the Σ -protocol.

1 Introduction

The breakthrough of Cramer, Damgård and Schoenmakers in [19] provided a technique for the OR-composition of Σ -protocols that has constantly received great interest in the last 30 years. While there have been several other proposals to perform OR-composition for specific requirements/applications (e.g., multi-round protocols [2, 1], Dlog-like languages [29, 4], delayed-input proofs [16, 17]), there is still an interesting open question that has found only partial answers recently: designing a non-interactive proof of knowledge of disjunctions through a composition applicable to a large class of Σ -protocols maintaining statistical zero-knowledge as in [19], but with sub-linear communication and minimizing the additional assumptions.

The OR-composition of [19] with the FS transform is particularly appealing since it (1) preserves the statistical zero-knowledge property of the underlying Σ -protocol and (2) requires no additional complexity assumptions. However, its communication complexity unfortunately grows linearly with the number n of statements in the disjunction.

⁴ Security against a malicious prover is limited to efficient ones, and technically the term “argument” is more precise, but we will still use the word proof for simplicity.

Stacking Sigmas. Interestingly, in [25], the authors proposed a proof of knowledge starting with a very large class of Σ -protocols, referred to as stackable Σ -protocols. The proof length is only logarithmic in the number of instances of the disjunction. With the FS transform, they obtain a statistical NIZK but, unfortunately, their result requires the discrete logarithm (Dlog) assumption.

NAPs. An alternative direction providing a different partial solution to the above open problem has been presented very recently in [12] by Boudgoust and Simkin. Their approach is based on non-adaptively programmable functions (NAPs)⁵. The NIZK obtained with the FS transform just relies on the random oracle but, unfortunately, it enjoys computational ZK only. The Σ -protocols that can be used in their construction must satisfy a definition of HVZK [8], which they observe to be equivalent to (challenge-independent) stackable Σ -protocols.

The need of idealized models and other approaches. Obtaining compact non-interactive proofs is known to be impossible unless one relies on idealized models or non-falsifiable assumptions [23]. The above works of [19, 25, 12] can rely on the widely studied FS transform to end up with a compact proof in the random oracle (RO) model.

Other approaches to non-interactive compact proofs leverage either idealized models model [38, 9, 43, 22] and/or non-falsifiable assumptions [40, 28], but they are out of the scope of this work since they do not consist of efficient transformations starting from a Σ -protocol. In other words, with just a restricted access (i.e., black-box without rewinds) to a prover executing a Σ -protocol, those constructions do not produce a compact non-interactive proof of disjunctions, which is instead possible using the results of [19, 25, 12]. Building a larger protocol by just having access to a prover of a Σ -protocol is interesting not only theoretically but also in practice. Indeed, one might leverage a device running the prover of a Σ -protocol in a trusted execution environment, to end up with a compact privacy-preserving proof building on top of the messages computed by the prover of the Σ -protocol.

1.1 Our Contribution

In this work, we solve the above open problem of constructing a compact statistical NIZK PoK for disjunctions in the RO model without any additional hardness assumption and having only a restricted access to a prover of a Σ -protocol. The class of Σ -protocols that we consider is stackable Σ -protocols where the HVZK property is statistical. This class encompasses not only the Σ -protocols modeled by the general frameworks of Cramer and Damgård [18] and Maurer [37] (which include well-known examples like [44, 30, 15]), but also includes⁶ Blum’s Hamiltonicity protocol [10] and the Goldreich, Micali, and Wigderson’s protocol for graph 3-coloring [27].

Our construction achieves statistical ZK, therefore protecting the prover’s secret with everlasting security. Our setting can be instantiated assuming that a cryptographic hash function H models a RO. The restricted access to the prover of the Σ -protocol consists of simply invoking it with two queries to obtain the first and the third message.

The communication complexity of our proof is $O(\lambda^2 \log n) + \text{CC}(\Sigma)$ where λ is the security parameter and $\text{CC}(\Sigma)$ is the communication of a single run of the underlying Σ -protocol. For

⁵ Such functions are introduced in [12] and can be seen as pseudorandom functions which allow for re-programming the output at an input point which must be fixed during key generation.

⁶ Recall that Σ -protocols can be instantiated with statistically hiding commitments in the RO model or with a setup.

example, when applying our composition to n instances of Blum’s Hamiltonicity protocol [10], each with m nodes, the total communication is $O(\lambda^2 \log n + m^2 \lambda^2)$. This is only a multiplicative overhead in the security parameter compared to the $O(\lambda \log n + m^2 \lambda^2)$ achieved by [25], which relies on number-theoretic assumptions. Notably, in sufficiently large graphs, which are interesting graphs to prove Hamiltonicity on, the dominant cost term is $m^2 \lambda^2$ in both approaches.

Furthermore, our approach yields significant communication improvements over the work of Boudgoust and Simkin [12], which lacks everlasting security. For example, considering a disjunction with n instances of the graph 3-coloring protocol [27], where each instance has m nodes and t edges, and considering a negligible soundness error, we obtain a communication complexity of $O(\lambda^2 \log n + tm \lambda^2)$, while [12] incurs a communication cost of $O(tm \lambda^4 \log n)$. We defer to App. B for a more detailed analysis.

1.2 Technical Overview

The approach of [25] combines stackable Σ -protocols with one main technical tool: a vector commitment scheme that is binding only in one position, while it is equivocal in all the others. Such special commitments were already known in the literature [39, 17], but notably [25] proposed a Dlog-based construction with perfect hiding that plays a fundamental role in preserving the statistical HVZK of the underlying Σ -protocol.

In order to circumvent the obstacles that limited prior work, we first revisit the core idea of [25] with the goal of reducing the requirements on the involved tools as much as possible, focusing on obtaining a 3-round statistical HVZK proof of knowledge (not necessarily a special-sound construction). We therefore reformulate the main idea of [25] so that it can still work on top of an easier-to-instantiate new tool that we define and name 1-out-of- n stackable commitment ((1, n)-SC). In the partially-binding vector commitment of [25], both the commit and the opening phase are non-interactive. Therefore, it is unfeasible for a malicious committer to produce a commitment and then open it to two vectors of messages that are distinct in all positions. In our case instead, while we keep the commit phase non-interactive, we introduce an interactive two-round opening phase. We observe that we can plug in a reformulation of the disjunctive composition technique of [25] the 2-round opening phase of our commitment, without increasing the total number of rounds and still obtaining a 3-round statistical HVZK PoK.

The opening phase of our commitment phase consists in the sender’s opening being computed upon receiving a random challenge from the receiver. We then require that, except with negligible probability, no adversarial sender can produce two accepting openings for two *different and random* challenges in which all opened messages differ between the two openings. The fact that *both* challenges are unpredictable is crucial for the binding of our commitment to hold. In particular, if the sender can pick the challenges, it can craft one commitment and a pair of *different* challenges so that it can successfully open to two vectors differing in all positions. One might think that this is inherent in commitments with interactive opening phase since knowing the challenge in advance should obviously allow the sender to cheat. However, there are constructions such as the one in [17] that have binding properties akin to the special soundness of Σ -protocols: it is possible to craft a commitment for one specific challenge c_0 s.t. not even a single message is fixed at committing time, however, it does not exist a second challenge $c_1 \neq c_0$ allowing to open the same commitment to a vector different to the one opened with challenge c_0 in all positions. Our commitment does not have this property which instead is trivially satisfied by the non-interactive commitment of [25]. Looking

ahead this makes the whole protocol not special-sound anymore but allows to rely on a easier-to-instantiate tool. Our reformulation of the main idea of [25] shows that their partially-binding vector commitment is in fact stronger than what is needed for the goal of stacking Σ -protocols to obtain a proof of knowledge. Therefore, we leverage our findings by defining our simpler 1-out-of- n stackable commitment $(1, n)$ -SC, which, through our reformulation of [25], suffices for obtaining a compact statistical HVZK 3-round proof of knowledge. We now give more details about our new special commitment scheme and how we construct it by only relying on a random oracle.

1.3 1-out-of- n Stackable Commitment Scheme $(1, n)$ -SC

At a high-level, after revisiting the core idea of [25] we minimize the requirements on the underlying commitment scheme, allowing also the opening to be interactive.

More precisely, we relax the opening phase so that it admits also a challenge from the receiver, followed by the actual opening of the sender, and we require the following properties: (a) *1-Binding*: After the commitment phase, there is always at least one position at which it is infeasible, except with negligible probability, for any adversarial sender to produce two accepting openings, for two different random challenges, that successfully open that position to two distinct messages. (b) *Extended Honest-Receiver Hiding*: the entire transcript, including commitment and opening phases, is indistinguishable from the output of an efficient simulator that gets as input the challenge and the opening, but has no information on the binding position (i.e., the binding position is kept hidden).

Construction of our 1-out-of- n stackable commitment $(1, n)$ -SC. We construct a $(1, n)$ -SC from a random oracle H in 3 steps. First, we design a $(1, 2)$ -SC, which we realize through another special bit-commitment scheme that we call $(1, 2)$ -bitSC. Next, we upgrade $(1, 2)$ -SC into a full-fledged $(1, n)$ -SC applying the general transformation given in Sec. 5.3. of [24], with careful adjustments due to our interactive opening phase.

Let us start by describing how we realize $(1, 2)$ -bitSC. At a high level, we aim to construct a commitment scheme that allows committing to two bits, m_0 and m_1 , such that one of them - specifically m_β - is fixed in position β during the commitment phase and thus it is supposed to be opened as m_β in position β , while the other bit, $m_{1-\beta}$, can be freely chosen at the time of opening. The commitment phase of $(1, 2)$ -bitSC uses a statistically hiding commitment SH (implied by the RO) and works as follows. For simplicity, let wlog $\beta = 0$. The sender, for committing to m_0 computes 3 commitments, d_0^0, d_1^0, d_2^0 where 2 of them, namely d_0^0, d_1^0 are commitments of the same random bit $\tilde{b}$, while d_2^0 commits to $v_0 = \tilde{b} \oplus m_0$. The sender computes one more time 3 commitments d_0^1, d_1^1, d_2^1 that are used for the equivocation - in the opening phase - of m_1 . In this case the sender chooses at random to commit to 0 in one of the first 2 commitments, and to 1 in the other commitment, therefore correlating d_0^1, d_1^1 , while d_2^1 is a commitment to a random bit. The opening of such commitment can be defined as the opening of one out of d_0^b and d_1^b (at random in the triple where m_β is committed, and selected appropriately in the triple where the opening is equivocal) and the opening of the third commitment d_2^b , for $b \in \{0, 1\}$. The final opened message is defined as a pair of messages, the first one is the XOR of the two opened messages for $b = 0$ and the second one is the XOR of the two opened messages for $b = 1$. One can easily see that in the above example where $\beta = 0$, the opening for $b = 1$ can be arbitrarily decided by the sender since d_0^1, d_1^1 are commitments to different bit messages.

Obviously, there is a very fragile binding here since a malicious sender could prepare all commitments so that the equivocation procedure is applicable to both triplets. We can mitigate such risks

as follows. The sender runs twice the above commitment procedure, therefore obtaining two tuples of six commitments each. One tuple will be defining the committed messages and therefore will be opened precisely as discussed above, while the other tuple will be used to check well-formedness (i.e., to check that at least one of the two tuples has been computed in a way that guarantees that its two triplets cannot correspond both to equivocal commitments). This is obtained by having the receiver send in the opening phase a random challenge bit, determining which one of the two tuples will be used for checking well-formedness and which one for opening. As a result, this scheme guarantees that with probability at least $1/2$ the equivocation procedure is applicable to at most one of the triplets. Finally, to hide the binding position β , the sender adds to each tuple of six commitments also a commitment, namely s^b , of a bit which specifies whether a swap of the two triplets of commitments should be done or not at opening time, therefore preventing adaptive selection of the binding position. Such a swap bit is opened only for the tuple used to verify the committed messages, and not for the tuple used to check well-formedness. Since the swap bits in the two tuples are independently sampled, the well-formedness check just reveals that at least one position is binding without revealing the actual binding position.

To build $(1, 2)$ -SC to commit to a string of λ bits, one could invoke λ times in parallel $(1, 2)$ -bitSC, i.e., one time for each bit of the message. However, this solution does not trivially imply a string commitment; this is because: **(a)** Each invocation of $(1, 2)$ -bitSC can be generated in such a way that one of the tuples of $(1, 2)$ -bitSC has both triplets of commitments in the equivocal mode (this goes undetected if the cut-and-choose did not select such tuples); **(b)** The adversary has the freedom to mix the binding position across different invocations of $(1, 2)$ -bitSC (and also between the two tuples of $(1, 2)$ -bitSC). This is because to guarantee the hiding of the binding position, the order of the triplets within each tuple must be selected independently at random.

Generally, one could mitigate the attack **(b)** by using an additional proof, such as an additional cut-and-choose, ensuring that the adversary changes the binding position only in very few locations, but this would degrade the performance of the commitment, incurring most likely in a cubic communication overhead in the security parameter.

We aim for a solution with at most quadratic overhead. To this end, we adopt a different approach that leverages the properties of H . Our proposal is a simple yet powerful modification: instead of restricting attack locations, we render such attacks ineffective by first hashing the message with H to obtain an l -bit digest, then committing bit-by-bit to this digest using $(1, 2)$ -bitSC. Looking ahead, this significantly complicates attacks: the sender now commits to a digest rather than the raw message, so successfully committing to two different messages requires finding two distinct digests that agree on the (mixed) binding positions.

This approach preserves the ability to equivocate in one position, as required. The only difference is that, instead of equivocating to the original message $m_{1-\beta}$, the sender now equivocates to its hash.

We focus now on explaining why this $(1, 2)$ -SC has the desired binding property. Namely, once the commitment $\text{com}_{(1,2)\text{-SC}}$ is generated there is always at least one out of the two position (i.e. position 0 or position 1) where it is infeasible, except with negligible probability, for any adversarial sender to produce two accepting openings, for two different random challenges, that successfully open that position to two distinct messages.

First, we notice that repeating $(1, 2)$ -bitSC l times in parallel ensures that in $\text{com}_{(1,2)\text{-SC}}$ only a small number of $(1, 2)$ -bitSC instances allow the equivocation procedure to be applied to both triplets of the opened tuple. Indeed, if a tuple is generated so that both triplets are in equivocal mode, this malicious behavior is detected with probability $1/2$. Therefore, if the adversarial sender

generates more than $\log^2 l$ (1, 2)-bitSC instances of this form, at least one of them will be checked by the receiver with overwhelming probability⁷.

From this argument follows also that once $\text{com}_{(1,2)\text{-SC}}$ has been generated, the adversarial sender must open consistently (i.e., to the same bit) in all locations in which the *same tuple* of (1, 2)-bitSC is required to be opened by the two different random challenges, except for at most $\log^2 l$ of such locations. More precisely, after $\text{com}_{(1,2)\text{-SC}}$ is fixed, the adversary opens the same bit either in position 0 or 1, in all the locations where the two challenges coincide, except possibly in at most $\log^2 l$ of them.

This analysis unfortunately does not prevent the adversarial sender to execute l independent runs of (1, 2)-bitSC with different binding positions. However, there are at most two possible binding positions (0 or 1). Hence, by the pigeonhole principle, the adversarial sender must reuse the same binding position in at least half of the runs of (1, 2)-bitSC; otherwise, there would exist a larger fraction of runs of (1, 2)-bitSC in which the binding position coincides.

Summing up, we have this two facts: (a) after $\text{com}_{(1,2)\text{-SC}}$ is fixed, for any pair of challenges, the adversary opens the same bit (either for position 0 or 1) in all locations where the challenges coincide, except possibly in at most $\log^2 l$ locations. Since the challenges are random, by the Hoeffding's bound, we know that this occurs with overwhelming probability in at least $l/4 - \log^2 l$ locations. Let us call S the set of those locations. (b) Once $\text{com}_{(1,2)\text{-SC}}$ is fixed, there exists at least s bits such that, for at least one of the two messages, such bits remain the same across two different openings. Furthermore, $s = |S|/2 = l/8 - \log^2 l$.

We now observe that a successful adversary should open to $m_0^0 \neq m_0^1$ and $m_1^0 \neq m_1^1$ s.t. $h_0^0 = H(m_0^0)$, $h_0^1 = H(m_0^1)$ or $h_1^0 = H(m_1^0)$ and $h_1^1 = H(m_1^1)$. Because of the above conclusions we have that either h_0^0 and h_0^1 or h_1^0 and h_1^1 coincide in at least $l/8 - \log^2(l)$ locations. Without loss of generality, let us assume that h_0^0 and h_0^1 coincide in $l/8 - \log^2(l)$ locations while $m_0^0 \neq m_0^1$. We observe that this can only happen with negligible probability in the random oracle model: the adversary must find two evaluations of H , namely h_0^0 and h_0^1 , that agree on at least s randomly selected bits, and $s = l/8 - \log^2(l)$ which is polynomial in the security parameter.

Revisiting the idea of [25]. We now give a high-level overview of the OR-composition of [25] to then illustrate how we can replace their partially-binding vector commitment with our (1, n)-SC.

Overview of [25]. Let $\beta \in [n]$ be the position for which the prover holds a witness for one of the n statements $x_1, \dots, x_n$ in the disjunction. In [25], the prover commits, using the partially-binding vector commitment, to the first-round message a_β obtained running the underlying Σ -protocol for x_β and sends the commitment to the verifier. The prover, after receiving the challenge from the verifier, computes the third-round message z_β running the underlying Σ -protocol for x_β and then it exploits the property of the stackable Σ -protocols to compute its third round as detailed next.

Specifically, stackable Σ -protocols feature an extended *deterministic* simulator that takes the statement, the challenge, and the third round and outputs the accepting first round. Furthermore, roughly, any z from the space of plausible third-round messages is compatible with any statement $x \in \mathcal{L}$. Thus the prover can re-use z_β over all statements, running the extended simulator to get all the accepting first-round messages $\{a_i\}_{i \in [n] \setminus \{\beta\}}$ for the other instances. At this point, the prover uses the partially binding vector commitment to equivocate all the positions $i \in [n] \setminus \{\beta\}$ to the a_i

⁷ The formal argument is a bit more involved as it requires relying on the computational binding of SH, while the above reasoning implicitly relies on statistical binding.

obtained from the simulator and sends the opening op and z_β to the verifier. Since the simulator is deterministic, the verifier can recompute all the underlying first-round messages running again the extended simulator on input z_β and then verify that they open the commitment received in the first round from the prover. Finally, the verifier just runs the verifier of the underlying Σ -protocol for all n instances and accepts iff all runs are accepting. The resulting protocol is compact because only one third-round message of the underlying Σ -protocol is sent and because the opening of the general transformation for partially binding commitments in Sec. 5.3. of [24] is succinct. Intuitively, the perfect hiding of the commitment preserves the statistical HVZK property while its computational binding leads to a computational variation of the special soundness property. In a nutshell, since the commitment is binding in at least one position, two accepting transcripts for the disjunction with the same first-round message (i.e., the same commitment) and different challenges will correspond to at least one of such a pair of transcripts of the underlying Σ -protocol, on which one can invoke the special soundness extractor.

Using $(1, n)$ -SC in a reformulation of the OR-composition of [25]. We now illustrate now how to replace the partially binding commitment of [25] with our less demanding $(1, n)$ -SC in the above template. There are a few main challenges to address: (1) The opening of $(1, n)$ -SC is interactive, unlike the partially binding commitment used in [25]; (2) $(1, n)$ -SC has a weak form of binding.

To address the first point, we revisit the construction of [25] s.t. the verifier will now send to the prover two sub-challenges, one for the Σ -protocols and one for the commitment scheme $(1, n)$ -SC itself. During the third round, the prover will open the commitment, taking into account the additional challenge received from the verifier. To prove statistical HVZK we introduced, and provided in our $(1, n)$ -SC, a deterministic simulator (called EHRH simulator) for the commitment scheme that operates analogously to the simulators of stackable Σ -protocol. The EHRH simulator deterministically re-computes the commitment given only the messages and the openings, but does not reveal the binding position.

Statistical HVZK is proven by showing an extended simulator for the disjunction protocol. Given the challenges, the third-round message, and the opening, one can run the simulator of the Σ -protocol n times to obtain the first-round messages and then the EHRH simulator of $(1, n)$ -SC to obtain the commitment (i.e., the first-round message of the disjunction protocol).

To ensure that our compiled construction preserves the proof of knowledge (PoK) property, we present a fundamentally different analysis from that of [25]. This is motivated by our goal of designing a primitive with minimal requirements, allowing the protocol to remain functional even under very weak binding guarantees. Proving the proof of knowledge property is unfortunately more challenging and the analysis of [25] does not apply. The main reason is that one cannot simply prove a computational variation of special soundness for our disjunction protocol as the binding of our $(1, n)$ -SC is weaker and only holds w.r.t. two openings w.r.t. *uniformly* distributed challenges; while special soundness holds for any possible pair of challenges. We therefore take a different path and prove the standard notion of proof of knowledge featuring an expected polynomial time rewinding extractor with oracle access to an adversarial prover.

FS transform and NIZK. Similarly to the constructions of [19, 25, 12], our 3-round public-coin argument system becomes a NIZK argument of knowledge when applying the FS transform, upgrading statistical HVZK to statistical ZK.

2 Related Work

In this paper, we focus on providing a methodology that directly uses a Σ -protocol for $\mathcal{L}$ and turns it into a statistical NIZK proof of knowledge system Π_{OR} for $\mathcal{L}_{\text{OR}}$ without adding any number-theoretic assumption; that is starting with Π we want to reach Π_{OR} only requiring a random oracle and through a restricted access to a prover of Π , preserving statistical ZK.

Several prior works [11, 14, 3, 5, 20, 29, 4, 25, 26, 7, 45, 6] produce compact proofs for disjunctions relying on number-theoretic hardness assumptions.

Other approaches enable the construction of proof systems based solely on symmetric cryptographic primitives. Blum [10] and Goldreich, Micali, and Wigderson [27] showed proof systems for graph-related NP-complete problems that only rely on commitment schemes. Using a statistically hiding commitment scheme their proof systems are statistical HVZK. Similarly, the MPC-in-the-head paradigm by Ishai et al. [34] allows one to prove in 3 rounds arbitrary statements, relying only on commitments. However, these approaches, beyond the fact that do not get a strong compactness (i.e., communication is usually larger than polylogarithmic) do not work in the setting where one would like to build on top of the access to a prover of a Σ -protocol.

Another direction involves constructing NIZK from random oracles using probabilistically checkable proofs [36] or interactive oracle proofs [9, 43]. Furthermore, another line of work has focused on designing proof systems optimizing disjunctions of statements expressed as circuits [32, 33, 46]. These methods yield communication complexities that are polylogarithmic but, more importantly, such techniques fail in obtaining a transformation to build a proof for a disjunction just accessing a prover of a base Σ -protocol.

A notable exception is the recent work of [12], which relies on CRHFs that, however, inherently degrade the special-honest verifier zero-knowledge flavor of the input protocol. That is, regardless of whether the base proof system for $\mathcal{L}$ is statistical honest-verifier zero knowledge or not, the resulting proof system for $\mathcal{L}_{\text{OR}}$ is only computational (honest-verifier) zero knowledge.

1-out-of- n commitments. Several constructions of 1-out-of- n commitments (i.e., vector commitments to n elements, where 1 position is binding and the other $n - 1$ can be equivocated) have been proposed in the literature under different names and slightly different security properties w.r.t. the one defined in this paper. Previous constructions, generally had more stringent security notions w.r.t. the one identified in our work. We highlight the most relevant examples below. In [25], a statistically hiding non-interactive commitment scheme relying on the Dlog assumption and trusted Pedersen parameters is proposed. The same authors also showed that with the help of a hash function, any 1-out-of-2 commitment readily gives a 1-out-of- 2^q one [24] that features a compact opening. This transformation, as we show in Sec. 5, can be generalized to interactive commitment schemes. Other constructions did not rely on a setup and instead gave interactive commitment schemes. In [17] an interactive commitment scheme based on instance-dependent trapdoor commitments is proposed. Their commitment unfortunately is inherently only computationally hiding and relies on number-theoretic assumptions. The work of [35] proposed a 1-out-of-2 commitment based on robust offline/online randomized encoding [31], which has a computational privacy notion that unfortunately leads to a computationally hiding commitment. Finally, in [39] a 1-out-of-2 commitment based on one-way functions is proposed. It is conceivable that by assuming CRHF instead, one could get a statistically hiding instantiation of their commitment. However, their commitment has a higher communication complexity w.r.t. what we construct in this work.

3 Preliminaries

We let $\mathbb{N}$ denote the set of all integers. For any integer $n \in \mathbb{N}$, denote by $[m]$ the set $\{1, 2, \dots, m\}$ and $\llbracket m \rrbracket$ the set $\{0, 1, \dots, m-1\}$. Vectors are denoted in bold font and are accessed using indices starting from 0, i.e., $\mathbf{v}[0]$ denotes the first element of vector $\mathbf{v}$. For any $x \in \{0, 1\}^*$ we denote $|x|$ as the number of bits representing x . For two machines M, A , we let $M^A(x)$ denote the output of machine M on input x and given oracle access to A . We denote by $x \leftarrow \$ X$ the process of uniformly choosing an element x in a set X . When writing $b \leftarrow A(x)$, we mean that b is the output of the probabilistic polynomial-time (PPT) algorithm A on input x . Furthermore, we denote with $A(x; r)$ an execution of the PPT algorithm A with fixed random coins r . Given two distributions D_1 and D_2 , we denote by $D_1 \approx_s D_2$ the fact that D_1 and D_2 are statistically indistinguishable.

3.1 Useful Bounds

Lemma 1 (Hoeffding's inequality). *Let $X_1, \dots, X_n$ be independent random variables with $a_i \leq X_i \leq b_i$ and let $S_n = X_1 + \dots + X_n$. Let $\mathbb{E}[S_n]$ denote the expected value of S_n . For every $t \geq 0$:*

$$\Pr[S_n - \mathbb{E}[S_n] \geq t] \leq e^{\frac{-2t^2}{\sum_{i=1}^n (b_i - a_i)^2}}.$$

Lemma 2. *Let $X_1, \dots, X_n$ iid random variables with a Bernoulli distribution with $p = 1/2$. It holds that:*

$$\Pr\left[\sum_{i=1}^n X_i \geq n/4\right] \leq \text{negl}(\lambda).$$

Proof. It follows by applying the Hoeffding's inequality with $t = n/2$.

3.2 Statistically Hiding Commitments

A non-interactive statistically hiding commitment **SH** is a pair of PPT algorithms (**Setup**, **Commit**) defined w.r.t. a message space $\mathcal{M}$ and a randomness space $\mathcal{R}$.

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: on input the security parameter λ , outputs some public parameters pp . pp is considered an implicit input of **Commit**.
- $\text{com} \leftarrow \text{Commit}(m; r)$: on input a message $m \in \mathcal{M}$ and a randomness $r \in \mathcal{R}$, outputs a commitment com .

Intuitively, a commitment satisfies two properties called binding and hiding. The binding property says that it is hard to open a commitment in two different ways. The hiding property says that a commitment hides the underlying message.

Definition 1 (Computational Binding). *We say that a non-interactive commitment is computationally binding if for all PPT adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that:*

$$\Pr\left[\text{Commit}(m_0; r_0) = \text{Commit}(m_1; r_1) \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ (m_0, m_1, r_0, r_1) \leftarrow \mathcal{A}(\text{pp}) \end{array}\right] \leq \text{negl}(\lambda).$$

Definition 2 (Statistical Hiding). *We say that a non-interactive commitment is statistically hiding if for all $m_0, m_1 \in \mathcal{M}$:*

$$\{\text{Commit}(m_0)\} \approx_s \{\text{Commit}(m_1)\}.$$

In this paper we use the following widely known computationally binding and statistically hiding commitment scheme assuming access to a random oracle H with output digest in $\{0, 1\}^\lambda$:

- $\text{com} \leftarrow \text{Commit}(m; r)$: given $m \in \{0, 1\}^*$ and $r \in \{0, 1\}^\lambda$, output $H(m||r)$.

3.3 Interactive Proofs Systems

We give here the definition of interactive proof systems. Given a pair of interactive Turing machines, P and V , we denote by $\langle P(w), V \rangle(x)$ the random variable representing the (local) output of V when interacting with machine P , where w is a private input for P , and x is a common input for P and V . The random input to each machine is uniformly and independently chosen.

Definition 3 (Interactive Proof System). *A pair of interactive machines (P, V) is called an interactive proof system for a language $\mathcal{L}$ if machine V is polynomial-time and the following two conditions hold:*

- *Completeness: For every $(x, w) \in R_{\mathcal{L}}$,*

$$\Pr[\langle P(w), V \rangle(x) = 1] \geq 1 - \text{negl}(\lambda).$$

- *Soundness: For every $x \notin \mathcal{L}$, and every interactive machine B ,*

$$\Pr[\langle B, V \rangle(x) = 1] \leq \text{negl}(\lambda).$$

In case that the soundness condition is required to hold only with respect to a computationally bounded prover, the pair (P, V) is called an interactive argument system.

An interactive proof system is informally considered a proof of knowledge if the prover not only convinces the verifier of a statement's validity but also demonstrates possession of a witness for it. This concept is formalized through the introduction of a machine E , known as the knowledge extractor. As its name suggests, the extractor E is designed to retrieve a witness from any dishonest prover P^* that successfully convinces an honest verifier. We define the proof of knowledge property as in [41].

Definition 4 (Proof of Knowledge). *Let (P, V) be an interactive proof system for the language $\mathcal{L}$ with witness relation $R_{\mathcal{L}}$. We say that (P, V) is a proof of knowledge if there exists a polynomial q and a probabilistic oracle machine E , such that for every PPT interactive machine P^* , for every $x \in \mathcal{L}$ and every $y, r \in \{0, 1\}^*$ such that $\Pr[\langle P_{x,y,r}^*, V(x) \rangle = 1] > 0$ where $P_{x,y,r}^*$ denotes the machine P^* with common input fixed to x , auxiliary input fixed to y and random tape fixed to r , the following holds*

1. *The expected number of steps taken by $E^{P_{x,y,r}^*}$ is bounded by*

$$\frac{q(|x|)}{\Pr[\langle P_{x,y,r}^*, V(x) \rangle = 1]}$$

where $E^{P_{x,y,r}^}$ denotes the machine E with oracle access to $P_{x,y,r}^*$*

2. *Furthermore,*

$$\Pr[\langle P_{x,y,r}^*, V(x) \rangle = 1, (x, w) \notin R_{\mathcal{L}} \mid w \leftarrow E^{P_{x,y,r}^*}] \leq \text{negl}(|x|)$$

3.4 Σ -Protocols

A Σ -protocol Σ is an interactive proof system (Sec. 3.3). In particular, it is a 3-round public-coin protocol consisting of a tuple of PPT algorithms (P_0, P_1, V) . It is defined w.r.t. an NP language $\mathcal{L}$ with a poly-time relation $R_{\mathcal{L}}$, a challenge space $\mathcal{C}_{\Sigma}$, and a randomness space $\mathcal{R}_{\Sigma}$. The algorithms work as follows:

- $a \leftarrow P_0(x, w; r)$ on input statement x , witness w , randomness $r \in \mathcal{R}_{\Sigma}$, outputs a first-round message a .
- $z \leftarrow P_1(x, w, a, c, r)$ on input statement x , witness w , first-round message a , randomness r used in the first round, and challenge $c \in \mathcal{C}_{\Sigma}$, outputs the third-round message z .
- $1/0 \leftarrow V(x, a, c, z)$, on input statement x , first-round message a , challenge $c \in \mathcal{C}_{\Sigma}$, and third-round message z , outputs 1 to accept and 0 to reject.

A transcript (a, c, z) w.r.t. a statement x is called *accepting* if $V(x, a, c, z) = 1$.

Definition 5 (Σ -protocol). A 3-round public-coin protocol $\Sigma = (P_0, P_1, V)$, is a Σ -protocol for an NP language $\mathcal{L}$ with a poly-time relation $R_{\mathcal{L}}$ iff the following properties hold.

Completeness: For all x and w such that $(x, w) \in R_{\mathcal{L}}$ it holds that:

$$\Pr \left[V(x, a, c, z) = 1 \mid \begin{array}{l} r \leftarrow \$ \mathcal{R}_{\Sigma}; c \leftarrow \$ \mathcal{C}_{\Sigma}; \\ a \leftarrow P_0(x, w; r); \\ z \leftarrow P_1(x, w, a, c, r) \end{array} \right] = 1.$$

Special Soundness: There exists a PPT extractor Ext , such that on input x and 2 accepting transcripts $(a, c_0, z_0), (a, c_1, z_1)$ for x , where $c_0 \neq c_1$, it holds that:

$$\Pr[(x, w) \in R_{\mathcal{L}} \mid w \leftarrow \text{Ext}(x, a, c_0, z_0, c_1, z_1)] = 1.$$

Special HVZK (SHVZK): There exists a PPT algorithm Sim_{Σ} such that for all $c \in \mathcal{C}_{\Sigma}$, and all $(x, w) \in R_{\mathcal{L}}$:

$$\left\{ (a, c, z) \mid r \leftarrow \$ \mathcal{R}_{\Sigma}, a \leftarrow P_0(x, w; r), z \leftarrow P_1(x, w, a, c, r) \right\} \approx_s \left\{ (a, c, z) \mid (a, z) \leftarrow \text{Sim}_{\Sigma}(x, c) \right\}.$$

3.5 Stackable Σ -Protocols

Stackable Σ -protocols [25] are a strengthening of the standard Σ -protocols. In particular, they satisfy a property (stronger than SHVZK) called extended honest-verifier zero-knowledge (EHVZK). EHVZK requires the existence of an *extended simulator*, that given a statement, a challenge, and a third-round message sampled from the space of admissible third-round messages for the given statement and challenge, outputs a *unique* first-round message. As an additional property, they require that such distribution does not depend on the statement. Namely, any third-round message sampled from such a distribution must be *re-usable* across different statements, so that multiple first-round messages can be derived from the same third-round message. Goel et al. (Sec. 6.1.1 of [24]) showed that all Σ -protocols can be made EHVZK and that usually Σ -protocols already have recyclable third-round messages. If the stackable Σ -protocol requires a setup, we assume the public parameters pp to be an implicit input of all algorithms.

Definition 6 (EHVZK). Let $\Sigma = (P_0, P_1, V)$, be an Σ -protocol for an NP language $\mathcal{L}$. Σ is EHVZK if there exists a deterministic polynomial-time algorithm Sim_Σ such that for all $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $c \in \mathcal{C}_\Sigma$, and all $(x, w) \in R_\mathcal{L}$ there exists an efficiently samplable distribution $D_{\text{pp}, x, c_\Sigma}^{(z)}$ such that

$$\left\{ (a, c, z) \mid r \leftarrow \$ \mathcal{R}_\Sigma, a \leftarrow P_0(x, w; r), z \leftarrow P_1(x, w, a, c, r) \right\} \approx_s \left\{ (a, c, z) \mid z \leftarrow \$ D_{\text{pp}, x, c_\Sigma}^{(z)}, a \leftarrow \text{Sim}_\Sigma(x, c, z) \right\}$$

Definition 7 (Σ -protocol with recyclable third messages). Let $\Sigma = (P_0, P_1, V)$ be a Σ -protocol for an NP language $\mathcal{L}$, Σ has recyclable third messages if for every $c \in \mathcal{C}_\Sigma$, $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, and all $(x, w) \in R_\mathcal{L}$, there exists an efficiently samplable distribution $D_{\text{pp}, c_\Sigma}^{(z)}$ such that $D_{\text{pp}, c_\Sigma}^{(z)} \approx_s \{z \mid r \leftarrow \$ \mathcal{R}_\Sigma; a \leftarrow P_0(x, w; r); z \leftarrow P_1(x, w, c, r)\}$.

Definition 8 (Stackable Σ -protocol). A Σ -protocol $\Sigma = (P_0, P_1, V)$, is stackable if it is an EHVZK Σ -protocol and has recyclable third messages.

4 (1, n)-Stackable Commitments

A 1-out-of- n stackable commitment (1, n)-SC is a 3-round public coin protocol between a sender S and a receiver R . It is defined w.r.t. an integer $n \geq 2$, a message space $\mathcal{M}$, a randomness space $\mathcal{R}_{\text{com}}$, and a challenge space $\mathcal{C}_{\text{com}}$. It consists of the following PPT algorithms:

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: on input the security parameter λ , outputs some public parameters pp . pp is considered as implicit input of all other algorithms.
- $\text{com} \leftarrow \text{Commit}(\beta, m_\beta; r)$: on input a binding position $\beta \in \llbracket n-1 \rrbracket$, a binding message $m_\beta \in \mathcal{M}$, and random coins $r \in \mathcal{R}_{\text{com}}$, outputs the first-round message of the protocol com .
- $\text{op} \leftarrow \text{Open}(\beta, \text{com}, c_{\text{com}}, \mathbf{m}, r)$: on input the binding position used in the first round β , a first-round message com , a challenge $c_{\text{com}} \in \mathcal{C}_{\text{com}}$, a vector of messages $\mathbf{m}$ of size n , and the random coins used in the first round r , outputs an opening op .
- $0/1 \leftarrow \text{ComVfy}(\text{com}, c_{\text{com}}, \text{op}, \mathbf{m})$: on input a first-round message com , a challenge $c_{\text{com}} \in \mathcal{C}_{\text{com}}$, an opening op , a vector of messages $\mathbf{m}$ of size n , outputs 1 to accept or 0 to reject.

Definition 9 (Completeness). For all $\beta \in \llbracket n-1 \rrbracket$, and all $\mathbf{m} \in \mathcal{M}^n$:

$$\Pr \left[\text{ComVfy}(\text{com}, c_{\text{com}}, \text{op}, \mathbf{m}) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ r \leftarrow \$ \mathcal{R}_{\text{com}}; c_{\text{com}} \leftarrow \$ \mathcal{C}_{\text{com}}; \\ \text{com} \leftarrow \text{Commit}(\beta, \mathbf{m}[\beta]; r); \\ \text{op} \leftarrow \text{Open}(\beta, \text{com}, c_{\text{com}}, \mathbf{m}, r) \end{array} \right] = 1.$$

Definition 10 (1-Binding). (1, n)-SC is 1-binding if for all PPT adversaries $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ there exists a negligible function $\text{negl}(\cdot)$ such that:

$$\Pr \left[\begin{array}{l} \text{ComVfy}(\text{com}, c_{\text{com}}^0, \text{op}^0, \mathbf{m}^0) = 1; \\ \text{ComVfy}(\text{com}, c_{\text{com}}^1, \text{op}^1, \mathbf{m}^1) = 1; \\ \forall j \in \llbracket n-1 \rrbracket : \mathbf{m}^0[j] \neq \mathbf{m}^1[j] \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda); \\ (\text{com}, \text{st}) \leftarrow \mathcal{A}_0(\text{pp}); \\ (c_{\text{com}}^0, c_{\text{com}}^1) \leftarrow \$ \mathcal{C}_{\text{com}}^2; \\ (\text{op}^0, \text{op}^1, \mathbf{m}^0, \mathbf{m}^1) \leftarrow \mathcal{A}_1(\text{st}, c_{\text{com}}^0, c_{\text{com}}^1) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition 11 (Extended Honest-Receiver Hiding). $(1, n)$ -SC is extended honest-receiver hiding (EHRH) if there exists a deterministic polynomial-time algorithm SimC such that for all $\mathbf{m} \in \mathcal{M}^n$, all $\beta \in \llbracket n-1 \rrbracket$, $c_{\text{com}} \in \mathcal{C}_{\text{com}}$, and $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ there exists an efficiently samplable distribution $D_{\text{pp}, \mathbf{m}, c_{\text{com}}}^{(\text{op})}$ such that:

$$\left\{ (\text{com}, c_{\text{com}}, \text{op}) \mid r \leftarrow \mathcal{R}_{\text{com}}, \text{com} \leftarrow \text{Commit}(\beta, m_\beta; r), \text{op} \leftarrow \text{Open}(\beta, \text{com}, c_{\text{com}}, \mathbf{m}, r) \right\} \approx_s \left\{ (\text{com}, c_{\text{com}}, \text{op}) \mid \text{op} \leftarrow D_{\text{pp}, \mathbf{m}, c_{\text{com}}}^{(\text{op})}, \text{com} \leftarrow \text{SimC}(c_{\text{com}}, \mathbf{m}, \text{op}) \right\}.$$

Definition 12 (Recyclable Openings). $(1, n)$ -SC has recyclable openings if for all $c_{\text{com}} \in \mathcal{C}_{\text{com}}$ and $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, there exists an efficiently samplable distribution $D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ such that for all $\mathbf{m} \in \mathcal{M}^n$, and all $\beta \in \llbracket n-1 \rrbracket$:

$$D_{\text{pp}, c_{\text{com}}}^{(\text{op})} \approx_s \{ \text{op} \mid r \leftarrow \mathcal{R}_{\text{com}}; \text{com} \leftarrow \text{Commit}(\beta, m_\beta; r); \text{op} \leftarrow \text{Open}(\beta, \text{com}, c_{\text{com}}, \mathbf{m}, r) \}.$$

5 Constructing $(1, n)$ -SC

We construct a $(1, n)$ -SC in three steps, we first construct a weak version of $(1, 2)$ -SC for the binary message space that we call $(1, 2)$ -bitSC, and we construct it from the statistically hiding commitment from random oracles described in Sec. 3.2. We then lift $(1, 2)$ -bitSC to a full-fledged $(1, 2)$ -SC for strings. Finally, we give a generic transformation from a $(1, 2)$ -SC to a $(1, 2^q)$ -SC where q is an arbitrary integer greater than 1.

5.1 Constructing $(1, 2)$ -bitSC

In this section, we construct $(1, 2)$ -bitSC = (WCommit, WOpen, WComVerify). For a high-level description of the tool we refer the reader to the technical overview, now we give a formal description of $(1, 2)$ -bitSC that uses the statistically hiding computationally binding commitment scheme SH = (Commit, Open) from random oracles described in Sec. 3.2 and the function f_{swap} defined as follows:

f_{swap} on input $b_{\text{swap}} \in \{0, 1\}$ and the vector of indices $(0, 1, 2, 3, 4, 5)$ does the following:

1. If $b_{\text{swap}} = 0$ outputs the vector of indices $\pi = (0, 1, 2, 3, 4, 5)$.
2. if $b_{\text{swap}} = 1$ outputs the vector of indices $\pi = (3, 4, 5, 0, 1, 2)$.

We will use the notation π_i to denote the index in the i -th position in π with $i \in \llbracket 5 \rrbracket$.

The algorithms of $(1, 2)$ -bitSC that are WCommit, WOpen, WComVerify are described respectively in Fig. 1, Fig. 2, Fig. 3. The construction requires no setup as we instantiate SH using a random oracle.

Theorem 1. *If SH is a computationally binding statistically hiding commitment scheme, then $(1, 2)$ -bitSC = (WCommit, WOpen, WComVerify) enjoys the following properties: completeness, extended honest-receiver hiding, and recyclable openings.*

Proof. Completeness follows by inspection.

$\text{WCommit}(\beta, m_\beta; r)$:

1. Parse $r = (u^b, \text{bit}_0^b, \text{bit}_1^b, v_1^b, r_s^b, r_k^b)_{b \in \{0,1\}, k \in \llbracket 5 \rrbracket}$, where $u^b, \text{bit}_0^b, \text{bit}_1^b, v_1^b \in \{0,1\}$, and all the randomnesses r_k^b are from the randomness space of SH
2. $\forall b \in \{0,1\} \pi^b \leftarrow f_{\text{swap}}(u^b, \langle 0, 1, 2, 3, 4, 5 \rangle)$
3. Compute:
 - (a) $\forall b \in \{0,1\} s^b \leftarrow \text{SH.Commit}(u^b \oplus \beta; r_s^b)$
4. $\forall b \in \{0,1\}$ do the following steps:
 - (a) Compute $v_0^b = \text{bit}_0^b \oplus m_\beta$
 - (b) Compute the following: $\backslash \backslash$ Compute the commitment for the binding position indexed by β
 - i. $d_0^b \leftarrow \text{SH.Commit}(\text{bit}_0^b; r_{\pi_0^b}^b)$
 - ii. $d_1^b \leftarrow \text{SH.Commit}(\text{bit}_1^b; r_{\pi_1^b}^b)$
 - iii. $d_2^b \leftarrow \text{SH.Commit}(v_0^b; r_{\pi_2^b}^b)$
 - (c) Compute the following: $\backslash \backslash$ Compute the commitment for the equivocal position indexed by $1 - \beta$
 - i. $d_3^b \leftarrow \text{SH.Commit}(\text{bit}_1^b; r_{\pi_3^b}^b)$
 - ii. $d_4^b \leftarrow \text{SH.Commit}(1 - \text{bit}_1^b; r_{\pi_4^b}^b)$
 - iii. $d_5^b \leftarrow \text{SH.Commit}(v_1^b; r_{\pi_5^b}^b)$
5. Set $\forall b \in \{0,1\} \text{com}^b = (d_{\pi_i^b}^b)_{i \in \llbracket 5 \rrbracket}$
6. Output $\text{com} = (\text{com}^0, \text{com}^1, s^0, s^1)$

Fig. 1: Commitment Phase of our (1, 2)-bitSC.

Recyclable openings: We start by describing $\hat{\text{D}}_{\text{pp}, c_{\text{com}}}^{(\text{op})}$. Intuitively, $\hat{\text{D}}_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ computes a series of commitments honestly (i.e, the ones encoding the binding and equivocal position), and just the opening of some others (i.e, the ones used to define the value of the messages). More in details, it samples a random binding position and creates all the commitments of the two tuples that encode such binding position (i.e., $d_0^b, d_1^b, d_3^b, d_4^b$), it then samples the opening (i.e., the randomness) that will be used (later by the simulator) to generate the commitments d_2^b, d_5^b . Finally, it outputs relevant data in a way coherent to the challenge c_{com} . $\hat{\text{D}}_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ is formally described in Fig. 8 of App. C.

We note that the distribution of the openings induced by $\hat{\text{D}}_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ is identical to the distribution of the openings generated by the honest execution of (1, 2)-bitSC.

Extended honest-receiver hiding: We consider the distribution $\hat{\text{D}}_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ described above and the simulator WSim_{com} defined formally in Fig. 9 of App. C. Intuitively, the simulator takes as input op which contains the randomness to re-compute the commitments. Notice that WOpen (Fig. 2) outputs the randomness to open the commitments requested by c_{com} and re-outputs all the commitments that are not opened⁸. This allows to re-compute com from op .

⁸ This is similar to the transformation in Sec. 6.1.1 of [24] to make any Σ -Protocol EHVZK. See App A of [24] and Sec. 4.3 of [13] for examples of applying this transformation to the hamiltonicity and 3-coloring Σ -Protocols respectively.

$\text{WOpen}(\beta, \text{com}, c_{\text{com}}, m_\beta, m_{1-\beta}, r) :$

1. Parse $r = (u^b, \text{bit}_0^b, \text{bit}_1^b, v_1^b, r_s^b, r_k^b)_{k \in \llbracket 5 \rrbracket, b \in \{0,1\}}$
2. Parse com as $(\text{com}^0, \text{com}^1, s^0, s^1)$ s.t. $\text{com}^b = (\widetilde{d_i^b})_{i \in \llbracket 5 \rrbracket, b \in \{0,1\}}$
3. Compute $\pi^b \leftarrow f_{\text{swap}}(u^b, \langle 0, 1, 2, 3, 4, 5 \rangle)$, for $b \in \{0, 1\}$
 $\backslash\backslash$ For the commitments in $\text{com}^{c_{\text{com}}}$ compute the opening
4. Set $\text{op}_{\text{part}_0} = (u^{c_{\text{com}}} \oplus \beta, r_s^{c_{\text{com}}})$ $\backslash\backslash$ Prepare opening for swap bit.
5. Sample $b \leftarrow_{\$} \{0, 1\}$ and prepare op_β in the following way:
 $\backslash\backslash$ Prepare the binding opening for the message m_β .
 - If $b = 0$
 - Set $\text{op}_\beta = (\pi_0^{c_{\text{com}}}, \pi_1^{c_{\text{com}}}, (\text{bit}_0^{c_{\text{com}}}, r_{\pi_0^{c_{\text{com}}}}^{c_{\text{com}}}), r_{\pi_2^{c_{\text{com}}}}^{c_{\text{com}}}, \widetilde{d_1^{c_{\text{com}}}})$
 - If $b = 1$
 - Set $\text{op}_\beta = (\pi_1^{c_{\text{com}}}, \pi_0^{c_{\text{com}}}, (\text{bit}_0^{c_{\text{com}}}, r_{\pi_1^{c_{\text{com}}}}^{c_{\text{com}}}), r_{\pi_2^{c_{\text{com}}}}^{c_{\text{com}}}, \widetilde{d_0^{c_{\text{com}}}})$
6. Set $b = 0$ if $(v_1^{c_{\text{com}}} \oplus m_{1-\beta}) = \text{bit}_1^{c_{\text{com}}}$, set $b = 1$ otherwise. Prepare $\text{op}_{1-\beta}$ in the following way:
 $\backslash\backslash$ Prepare the equivocal opening for the message $m_{1-\beta}$.
 - If $b = 0$:
 - Set $\text{op}_{1-\beta} = (\pi_3^{c_{\text{com}}}, \pi_4^{c_{\text{com}}}, (\text{bit}_1^{c_{\text{com}}}, r_{\pi_3^{c_{\text{com}}}}^{c_{\text{com}}}), r_{\pi_5^{c_{\text{com}}}}^{c_{\text{com}}}, \widetilde{d_4^{c_{\text{com}}}})$
 - Else:
 - Set $\text{op}_{1-\beta} = (\pi_4^{c_{\text{com}}}, \pi_3^{c_{\text{com}}}, (1 - \text{bit}_1^{c_{\text{com}}}, r_{\pi_4^{c_{\text{com}}}}^{c_{\text{com}}}), r_{\pi_5^{c_{\text{com}}}}^{c_{\text{com}}}, \widetilde{d_3^{c_{\text{com}}}})$
7. Prepare $\text{op}_{\text{part}_3}$ in the following way
 $\backslash\backslash$ For the commitments in $\text{com}^{1-c_{\text{com}}}$ open the randomness of the binding position.
 - Set $\text{op}_{\text{part}_3} = (\pi_0^{1-c_{\text{com}}}, \pi_1^{1-c_{\text{com}}}, (\text{bit}_0^{1-c_{\text{com}}}, r_{\pi_0^{1-c_{\text{com}}}}^{1-c_{\text{com}}}), (\text{bit}_0^{1-c_{\text{com}}}, r_{\pi_1^{1-c_{\text{com}}}}^{1-c_{\text{com}}}), \{\widetilde{d_k^{1-c_{\text{com}}}}\}_{k \in \{2,3,4,5\}}, s^{1-c_{\text{com}}})$
8. If $u^{c_{\text{com}}} = 0$ set $\text{op}_{\text{part}_1} = \text{op}_\beta, \text{op}_{\text{part}_2} = \text{op}_{1-\beta}$, else $\text{op}_{\text{part}_1} = \text{op}_{1-\beta}, \text{op}_{\text{part}_2} = \text{op}_\beta$
 $\backslash\backslash$ Prepare the openings in the right order.
9. Output $\text{op} = (\text{op}_{\text{part}_0}, \text{op}_{\text{part}_1}, \text{op}_{\text{part}_2}, \text{op}_{\text{part}_3})$

Fig. 2: Opening Phase of our (1, 2)-bitSC.

The commitments are recomputed by WSim_{com} based on the input bit messages m_0 and m_1 .

To prove indistinguishability between the honest execution of (1, 2)-bitSC and a simulated execution generated using $\hat{\text{D}}_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ and WSim_{com} we consider the following hybrids:

$\mathcal{H}_0$: It is the distribution of $(\text{com}, c_{\text{com}}, \text{op})$ induced by a regular execution of (1, 2)-bitSC w.r.t. inputs $\beta, m_\beta, m_{1-\beta}$.

$\mathcal{H}_1$: It is identical to $\mathcal{H}_0$ except that com is generated by sampling the opening $\text{op}^* \leftarrow_{\$} \hat{\text{D}}_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ and running $\text{WSim}_{\text{com}}(c_{\text{com}}, m_0, m_1, \text{op})$ instead of the honest committing procedure.

This proof is concluded noticing that $\mathcal{H}_0$ and $\mathcal{H}_1$ are statistically close due to the statistical hiding of Π_{SH} and the perfect security of One-Time Pad.

Communication complexity. The communication complexity of our (1, 2)-bitSC is $|\text{com}| + |c_{\text{com}}| + |\text{op}| = 6|\text{com}_{\text{SH}}| + 1 + 6|\text{op}_{\text{SH}}|$, where $|\text{com}_{\text{SH}}|$ is the size of the commitment in SH, and $|\text{op}_{\text{SH}}|$ is the

WComVerify($\text{com}, c_{\text{com}}, \text{op}, m_0, m_1$):

1. Parse com as $(\text{com}^0, \text{com}^1, s^0, s^1)$ s.t.
 $\text{com}^b = (d_0^b, d_1^b, d_2^b, d_3^b, d_4^b, d_5^b)$, $b \in \{0, 1\}$
2. Parse op as $(\text{op}_{\text{part}_0}, \text{op}_{\text{part}_1}, \text{op}_{\text{part}_2}, \text{op}_{\text{part}_3})$ with:
 $\text{op}_{\text{part}_0}: b_{\text{swap}}, r_s^{c_{\text{com}}}$
 $\text{op}_{\text{part}_1}: i_0, i_1, \text{bit}_0^{c_{\text{com}}}, r_{i_0}^{c_{\text{com}}}, r_2^{c_{\text{com}}}, e_{i_1}^{c_{\text{com}}}$
 $\text{op}_{\text{part}_2}: k_0, k_1, \text{bit}_1^{c_{\text{com}}}, r_{k_0}^{c_{\text{com}}}, r_5^{c_{\text{com}}}, e_{k_1}^{c_{\text{com}}}$
 $\text{op}_{\text{part}_3}: l_0, l_1, (\text{bit}_0^{1-c_{\text{com}}}, r_{l_0}^{1-c_{\text{com}}}), (\text{bit}_0^{1-c_{\text{com}}}, r_{l_1}^{1-c_{\text{com}}}), \{\widetilde{e_k^{1-c_{\text{com}}}}\}_{k \in \{2,3,4,5\}}, \widetilde{s^{1-c_{\text{com}}}}$
3. Compute the following steps:
 $\backslash\backslash$ For the commitment in $\text{com}^{c_{\text{com}}}$ check the openings of the commitment.
 (a) Output 0 if $s^{c_{\text{com}}} \neq \text{SH.Commit}(b_{\text{swap}}; r_s^{c_{\text{com}}})$
 (b) Output 0 if one of the checks fails:
 i. $i_0, i_1 \in \{0, 1\} \wedge i_0 \neq i_1 \wedge k_0, k_1 \in \{3, 4\} \wedge k_0 \neq k_1$
 ii. $\text{SH.Commit}(\text{bit}_0^{c_{\text{com}}}; r_{i_0}^{c_{\text{com}}}) = d_{i_0}^{c_{\text{com}}}$
 iii. $\text{SH.Commit}(v_0^{c_{\text{com}}}; r_2^{c_{\text{com}}}) = d_2^{c_{\text{com}}}$, where $v_0^{c_{\text{com}}} = m_{b_{\text{swap}}} \oplus \text{bit}_0^{c_{\text{com}}}$
 iv. $\text{SH.Commit}(\text{bit}_1^{c_{\text{com}}}; r_{k_0}^{c_{\text{com}}}) = d_{k_0}^{c_{\text{com}}}$
 v. $\text{SH.Commit}(v_1^{c_{\text{com}}}; r_5^{c_{\text{com}}}) = d_5^{c_{\text{com}}}$, where $v_1^{c_{\text{com}}} = m_{1-b_{\text{swap}}} \oplus \text{bit}_1^{c_{\text{com}}}$
 vi. $e_{i_1}^{c_{\text{com}}} = d_{i_1}^{c_{\text{com}}}$
 vii. $e_{k_1}^{c_{\text{com}}} = d_{k_1}^{c_{\text{com}}}$
4. Compute the following steps:
 $\backslash\backslash$ For the commitments in $\text{com}^{1-c_{\text{com}}}$ check that there is a binding position.
 (a) Output 0 if $\widetilde{s^{1-c_{\text{com}}}} \neq s^{1-c_{\text{com}}}$
 (b) Output 0 if one of the checks fails:
 i. $\text{SH.Commit}(\text{bit}_0^{1-c_{\text{com}}}; r_{l_0}^{1-c_{\text{com}}}) = d_{l_0}^{1-c_{\text{com}}}$
 ii. $\text{SH.Commit}(\text{bit}_0^{1-c_{\text{com}}}; r_{l_1}^{1-c_{\text{com}}}) = d_{l_1}^{1-c_{\text{com}}}$
 iii. $e_2^{1-c_{\text{com}}} = d_2^{1-c_{\text{com}}}$
 iv. $e_3^{1-c_{\text{com}}} = d_3^{1-c_{\text{com}}}$
 v. $e_4^{1-c_{\text{com}}} = d_4^{1-c_{\text{com}}}$
 vi. $e_5^{1-c_{\text{com}}} = d_5^{1-c_{\text{com}}}$

Output 1

Fig. 3: Verification of our (1, 2)-bitSC.

size of the opening in SH. Considering the statistically hiding commitment scheme SH of Sec. 3.2 we have that $|\text{com}_{\text{SH}}| = O(\lambda)$, and $|\text{op}_{\text{SH}}| = O(\lambda)$. Therefore, the resulting communication complexity of (1, 2)-bitSC is $O(\lambda)$.

5.2 Constructing (1, 2)-SC from (1, 2)-bitSC

In this section, we construct 1-out-of-2 stackable commitment scheme for strings (1, 2)-SC starting from our weaker tool (1, 2)-bitSC described in the previous section. For a high-level description

of the tool we refer the reader to the technical-overview, now we give a formal description of $(1,2)\text{-SC} = (\text{Commit}, \text{Open}, \text{ComVfy})$, that uses $(1,2)\text{-bitSC} = (\text{WCommit}, \text{WOpen}, \text{WComVerify})$, defined in Sec. 5.1, and a random oracle H with digest length l . The formal description of the algorithms Commit , Open , ComVfy is given in Fig. 4.

$\text{Commit}(\beta, m_\beta; r)$:

1. Parse r as $(r_i)_{i \in [l]}$
2. Compute $h_\beta \leftarrow H(m_\beta)$
3. For all $i \in [l]$:
 - (a) Compute $\text{com}_i \leftarrow \text{WCommit}(\beta, h_{\beta,i}; r_i)$, where $h_{\beta,i}$ denotes the i -th bit of h_β
4. Set $\text{com} = (\text{com}_i)_{i \in [l]}$
5. Output com

$\text{Open}(\beta, \text{com}, c_{\text{com}}, m_\beta, m_{1-\beta}, r) :$

1. Parse com as $(\text{com}_i)_{i \in [l]}$
2. Parse r as $(r_i)_{i \in [l]}$
3. $h_{1-\beta} \leftarrow H(m_{1-\beta})$
4. $h_\beta \leftarrow H(m_\beta)$
5. For all $i \in [l]$:
 - (a) Compute $\text{op}_i \leftarrow \text{WOpen}(\beta, \text{com}_i, c_{\text{com}}^i, h_{1-\beta,i}, h_{\beta,i}, r_i)$, where $c_{\text{com}}^i, h_{\beta,i}, h_{1-\beta,i}$ denotes the i -th bit of respectively $c_{\text{com}}, h_\beta, h_{1-\beta}$
6. Set $\text{op} = (\text{op}_i)_{i \in [l]}$
7. Output op

$\text{ComVfy}(\text{com}, c_{\text{com}}, \text{op}, m_0, m_1)$:

1. Parse com as $(\text{com}_i)_{i \in [l]}$
2. Parse op as $(\text{op}_i)_{i \in [l]}$
3. Let $\tilde{h}_0$ and $\tilde{h}_1$ be the messages committed in com and obtained from op .
4. Output 0 if, for $b \in \{0,1\}$, the following checks fail:
 - (a) $h_b \leftarrow H(m_b)$, $h_b \neq \tilde{h}_b$
5. Output 0 if the following checks fail:
 - (a) For all $i \in [l]$:
 - i. $\text{WComVerify}(\text{com}_i, c_{\text{com}}^i, \text{op}_i, \tilde{h}_{0,i}, \tilde{h}_{1,i}) = 1$ where $c_{\text{com}}^i, \tilde{h}_{1,i}, \tilde{h}_{0,i}$ is the i -th bit of respectively $c_{\text{com}}, \tilde{h}_0, \tilde{h}_1$
6. Output 1

Fig. 4: Commitment, Open, and Verification Phase of our $(1,2)\text{-SC}$.

Theorem 2. Let $(1, 2)\text{-bitSC} = (\text{WCommit}, \text{WOpen}, \text{WComVerify})$, be the scheme of Sec. 5.1, then $(1, 2)\text{-SC} = (\text{Commit}, \text{Open}, \text{ComVfy})$ is a 1-out-of-2 stackable commitment (Sec. 4) in the (non-programmable) ROM.

Proof. Completeness follows by inspection.

Recyclable openings: We start by describing $D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ that is obtained executing $\text{op}_i \leftarrow \$ \hat{D}_{\text{pp}, c_{\text{com}}}^{(\text{op})}$, for $i \in [l]$ and giving in output $(\text{op}_i)_{i \in [l]}$. We note that the distribution of the openings induced by $D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ is identically distributed to the distribution of the openings generated by the honest execution of $(1, 2)\text{-SC}$ due to the recyclable opening property of $(1, 2)\text{-bitSC}$.

Extended honest-receiver hiding: To prove this property we consider the distribution $D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ defined above and the simulator SimC described in Fig. 5. Informally, the simulator $\text{SimC}(c_{\text{com}}, m_0, m_1, \text{op})$ runs $\mathbf{h}_b \leftarrow \mathbf{H}(m_b)$, for $b \in \{0, 1\}$, and then it runs WSim_{com} for each single bit of $(\mathbf{h}_0, \mathbf{h}_1)$. To prove indistinguishability between the honest execution of $(1, 2)\text{-SC}$ and a simulated execution generated using $D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ and SimC we consider the following hybrids:

$\mathcal{H}_0$: It is the distribution of $(\text{com}, c_{\text{com}}, \text{op})$ induced by a regular execution of $(1, 2)\text{-SC}$ w.r.t. inputs $\beta, m_\beta, m_{1-\beta}$.

$\mathcal{H}_1$: It is identical to $\mathcal{H}_0$ except that com is generated by sampling the opening $\text{op}^* \leftarrow \$ D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ and running $\text{SimC}(c_{\text{com}}, m_0, m_1, \text{op})$ instead of the honest committing procedure.

The proof ends observing that the distribution of the output of $\mathcal{H}_1$ is statistically close the distribution of the output of $\mathcal{H}_0$ due to the EHRH property of $(1, 2)\text{-bitSC}$.

1-Binding: We show a reduction to the binding of the underlying non-interactive commitment scheme (used to construct $(1, 2)\text{-bitSC}$), demonstrating that the only way that the adversary can win the 1-Binding game with non-negligible probability is by guessing a sufficiently large fraction of a random oracle evaluation, which is unfeasible in polynomial time, thus leading to a contradiction. More formally, we proceed with the following lemmas.

Lemma 3. If $c_{\text{com}}^0, c_{\text{com}}^1$ are two uniformly random string of l -bits then, except with negligible probability, $|\mathcal{C}| > \frac{l}{4}$. Where $\mathcal{C}$ is the set of indices s.t. $\forall j \in \mathcal{C} \ c_{\text{com},j}^0 = c_{\text{com},j}^1$.

Proof. Since $c_{\text{com}}^0, c_{\text{com}}^1$ are uniformly distributed it must be the case that, except with negligible probability, there exists a set $\mathcal{C}$ of indices s.t. $\forall j \in \mathcal{C} \ c_{\text{com},j}^0 = c_{\text{com},j}^1$ and $|\mathcal{C}| > l/4$ due Lemma 2. To see this, let X_i be the random variable that assumes value 1 if two randomly sampled bits are equal and 0 otherwise. We have that this random variable is a bernoullian random variable with $p = 1/2$ and that $|\mathcal{C}| = \sum_{i=1}^l X_i$.

Lemma 4. If the adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ provides for the same commitment com two accepting openings op^0, op^1 , w.r.t. random challenges $c_{\text{com}}^0, c_{\text{com}}^1$, such that op^0 opens to m_0^0, m_1^0 (w.r.t. challenge c_{com}^0) and op^1 opens to m_0^1, m_1^1 (w.r.t. challenge c_{com}^1) s.t. $m_0^0 \neq m_0^1$ and $m_1^0 \neq m_1^1$, and $\mathbf{h}_{b'}^b \leftarrow \mathbf{H}(m_{b'}^b)$, $\forall b, b' \in \{0, 1\}$ then, except with negligible probability, $|\mathcal{S} \cap \mathcal{C}| \leq \log^2 l$. Where $\mathcal{S}$ is the set of indices s.t. $\forall i \in \mathcal{S} \ \mathbf{h}_{0,i}^0 \neq \mathbf{h}_{1,i}^0 \wedge \mathbf{h}_{0,i}^1 \neq \mathbf{h}_{1,i}^1$, $\mathcal{C}$ is the set of indices s.t. $\forall j \in \mathcal{C} \ c_{\text{com},j}^0 = c_{\text{com},j}^1$, and l is the bit-length of a output digest of $\mathbf{H}$.

Proof Intuition. Suppose by contradiction that $|\mathcal{S} \cap \mathcal{C}| > \log^2 l$, then, intuitively, the adversary must be breaking the binding of the underlying bit commitment SH used to construct $(1, 2)\text{-bitSC}$. In particular, the j -th invocation of $(1, 2)\text{-bitSC}$ is composed of the non-interactive commitments

$\{d_{i,j}^b\}_{i \in [5]}, s_j^b, b \in \{0, 1\}$. Unless of violating the binding of the non-interactive commitment SH, the only way for which $h_{0,j}^0 \neq h_{1,j}^0 \wedge h_{0,j}^1 \neq h_{1,j}^1$ is because both $d_{0,j}^{b'}$ and $d_{1,j}^{b'}$ and $d_{3,j}^{b'}$ and $d_{4,j}^{b'}$ are commitments of different bits for $b' = c_{\text{com},j}^0 = c_{\text{com},j}^1$ for each $j \in \{S \cap C\}$. However, unless of violating the binding of the non-interactive commitment SH, this occurs only with negligible probability.

To see this, first observe that as $\mathcal{A}$ is breaking 1-Binding with non-negligible probability p , it will also provide, with probability p , a pair of openings breaking the 1-Binding property when queried with fresh random challenges $c_{\text{com}}^\alpha, c_{\text{com}}^\beta$. Second, since $|S \cap C| > \log^2 l$, when asking the adversary to reply to a new pair of random challenges $c_{\text{com}}^\alpha, c_{\text{com}}^\beta$ then, except with negligible probability, there exists a $j \in S \cap C$ so that $c_{\text{com}}^\alpha \neq c_{\text{com}}^\beta$. As a result, both $d_{0,j}^{b'}$ and $d_{1,j}^{b'}$ or both $d_{3,j}^{b'}$ and $d_{4,j}^{b'}$ will be required to be opened and the opened pair will commit to the same bit, otherwise the verifier won't accept.

Proof. Suppose by contradiction that $|S \cap C| > \log^2 l$, and let $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ be the adversary that wins the 1-Binding experiment with non-negligible probability, then we are able to show an adversary $\mathcal{A}'$ that breaks the binding of the non-interactive commitment SH used to construct (1, 2)-bitSC.

Let us shortly recall that (1, 2)-SC is composed of l invocations of (1, 2)-bitSC and the j -th invocation of (1, 2)-bitSC is composed of multiple invocations of the non-interactive commitments SH, namely $\{d_{i,j}^b\}_{i \in [5]}, s_j^b, b \in \{0, 1\}$, for $j \in [l]$.

$\mathcal{A}'$ starts with activating $\mathcal{A}_0$ and obtaining the commitment (com, st) , then it computes the following steps:

1. Samples uniformly random challenge $c_{\text{com}}^0, c_{\text{com}}^1$ and runs $\mathcal{A}_1(\text{st}, c_{\text{com}}^0, c_{\text{com}}^1)$ obtaining op^0 w.r.t. messages m_0^0, m_1^0 and op^1 w.r.t. messages m_0^1, m_1^1 s.t. $h_{b'}^b \leftarrow H(m_{b'}^b), \forall b, b' \in \{0, 1\}$.
2. Compute the set $\{S \cap C\}$ using $(\text{op}^0, \text{op}^1)$ and $(c_{\text{com}}^0, c_{\text{com}}^1)$.
3. If there exists $j \in \{S \cap C\}$ s.t. $s_j^{b'}$ has an opening $\widetilde{b_{\text{swap}}^{b'}}; \widetilde{r_s^{b'}}$ in op^0 and a different opening $\widetilde{b_{\text{swap}}^{b'}}; \widetilde{r_s^{b'}}$ in op^1 then output $s_j^{b'}$, $\widetilde{b_{\text{swap}}^{b'}}; \widetilde{r_s^{b'}}$, $\widetilde{b_{\text{swap}}^{b'}}; \widetilde{r_s^{b'}}$ with $b' = c_{\text{com},j}^0 = c_{\text{com},j}^1$.
4. If there exists $j \in \{S \cap C\}$ and $i \in [5]$ s.t. $d_{i,j}^{b'}$ has an opening $\widetilde{v_i^{b'}}; \widetilde{r_i^{b'}}$ in op^0 and a different opening $\widetilde{v_i^{b'}}; \widetilde{r_i^{b'}}$ in op^1 , then output $d_{i,j}^{b'}$ and $\widetilde{v_i^{b'}}; \widetilde{r_i^{b'}}$, $\widetilde{v_i^{b'}}; \widetilde{r_i^{b'}}$, with $b' = c_{\text{com},j}^0 = c_{\text{com},j}^1$.
5. Samples uniformly random challenge $c_{\text{com}}^\alpha, c_{\text{com}}^\beta$ and runs $\mathcal{A}_1(\text{st}, c_{\text{com}}^\alpha, c_{\text{com}}^\beta)$ obtaining op^α and op^β .
6. If there exists $j \in \{S \cap C\}$ and $i \in [5]$ s.t. $d_{i,j}^{b'}$ has an opening $\widetilde{v_i^{b'}}; \widetilde{r_i^{b'}}$ in $\{\text{op}^0\}$ (or in op^1) and a different opening $\widetilde{v_i^{b'}}; \widetilde{r_i^{b'}}$ in op^α , then output $d_{i,j}^{b'}$ and $\widetilde{v_i^{b'}}; \widetilde{r_i^{b'}}$, $\widetilde{v_i^{b'}}; \widetilde{r_i^{b'}}$, with $b' = c_{\text{com},j}^0 = c_{\text{com},j}^1 \neq c_{\text{com},j}^\alpha$.
7. Output $\perp$.

We argue that $\mathcal{A}'$ terminates before step 7 with non-negligible probability.

Note that if $\mathcal{A}'$ did not terminate after step 4 then after this step $\mathcal{A}'$ collected all the openings of the commitments $d_{i,j}^{b'}$ with $j \in \{S \cap C\}$ and $i \in [5]$ and $b' = c_{\text{com},j}^0 = c_{\text{com},j}^1$. We observe that (a) $\mathcal{A}$ did not terminate in step 3; (b) by hypothesis $h_{0,j}^0 \neq h_{1,j}^0 \wedge h_{0,j}^1 \neq h_{1,j}^1$. It follows from observations (a) and (b) that $d_{0,j}^{b'}, d_{1,j}^{b'}$ are commitments of different bit-values and $d_{3,j}^{b'}, d_{4,j}^{b'}$ are commitments of different bit-values, for all $j \in \{S \cap C\}$. Then, unless of a negligible probability, there exist at least one index j^* s.t. $c_{\text{com},j^*}^0 = c_{\text{com},j^*}^1 \neq c_{\text{com},j^*}^\alpha$, this is because $|S \cap C| > \log^2 l$ and the challenges are randomly sampled. Therefore in step 5 $\mathcal{A}'$ collects a different opening

for one of the commitments $d_{i,j^*}^{b'}$. This is because by hypothesis $\mathcal{A}$ is breaking 1-binding with non-negligible probability and therefore is providing an accepting opening op^α where $d_{0,j^*}^{b'}$, $d_{1,j^*}^{b'}$ or $d_{3,j^*}^{b'}$, $d_{4,j^*}^{b'}$ commit to the same bit-value, where $b' = c_{\text{com},j^*}^0 = c_{\text{com},j^*}^1 \neq c_{\text{com},j^*}^\alpha$

We now proceed to reach the final contradiction.

It follows from Lemma 3 and Lemma 4 that If the adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ provides for the same commitment com two accepting openings op^0 , op^1 , w.r.t. challenges $c_{\text{com}}^0, c_{\text{com}}^1$, such that op^0 opens to m_0^0, m_1^0 (w.r.t. challenge c_{com}^0) and op^1 opens to m_0^1, m_1^1 (w.r.t. challenge c_{com}^1) s.t. $m_0^0 \neq m_1^0$ and $m_0^1 \neq m_1^1$, and $h_{b'}^b \leftarrow H(m_{b'}^b)$, $\forall b, b' \in \{0, 1\}$ then, except with negligible probability, $|S' \cap C| \geq l/4 - \log^2 l$. Where S' is the set of indices s.t. $\forall i \in S' \ h_{0,i}^0 = h_{1,i}^0 \vee h_{0,i}^1 = h_{1,i}^1$, C is the set of indices s.t. $\forall j \in C \ c_{\text{com},j}^0 = c_{\text{com},j}^1$, and l is the bit-length of a output digest of H .

Let S^b with $b \in \{0, 1\}$ the sets of indices such that $\forall i \in \{S' \cap C\} \ h_{0,i}^b = h_{1,i}^b$. Let S^{b^*} the set of maximum cardinality between S^0 and S^1 , then it holds that $s = |S^{b^*}| \geq l/8 - \log^2 l$. This means that $\mathcal{A}$, by only querying the random oracle a polynomial number of times, found with non-negligible probability two messages $m_0^{b^*} \neq m_1^{b^*}$ such that $h_0^{b^*} \leftarrow H(m_0^{b^*})$, $h_1^{b^*} \leftarrow H(m_1^{b^*})$ and digests $h_1^{b^*} h_0^{b^*}$ match in a polynomial fraction s of locations randomly selected by the challenges. However, we know that the probability of such an event is negligible in the non-programmable random oracle model, thus reaching a contradiction.

Communication complexity. The communication complexity of our (1,2)-SC is $|\text{com}| + |c_{\text{com}}| + |\text{op}| = l|\text{com}_{(1,2)\text{-bitSC}}| + |c_{\text{com}}| + l|\text{op}_{(1,2)\text{-bitSC}}|$, where $|\text{com}_{(1,2)\text{-bitSC}}|$ is the size of the commitment in (1,2)-bitSC, $|c_{\text{com}}| = l$, and $l|\text{op}_{(1,2)\text{-bitSC}}|$ is the size of the opening in (1,2)-bitSC. Considering our (1,2)-bitSC of Sec. 5.2 we have that $|\text{com}_{(1,2)\text{-bitSC}}| = O(\lambda)$, $|c_{\text{com}}| = l$, and $|\text{op}_{(1,2)\text{-SC}}| = O(\lambda)$. Therefore assuming that $l = O(\lambda)$, the communication complexity of (1,2)-SC is $O(\lambda^2)$.

SimC($c_{\text{com}}, m_0, m_1, \text{op}$) :

1. Parse op as $(\text{op}_i)_{i \in [l]}$
2. $h_0 \leftarrow H(m_0)$
3. $h_1 \leftarrow H(m_1)$
4. For all $i \in [l]$:
 - $\text{com}_i \leftarrow \text{WSim}_{\text{com}}(c_{\text{com}}^i, h_{0,i}, h_{1,i}, \text{op}_i)$ where $c_{\text{com}}^i, h_{0,i}, h_{1,i}$ is the i -th bit of respectively c_{com}, h_0, h_1
5. $\text{com} = (\text{com}_i)_{i \in [l]}$
6. Output com

Fig. 5: Simulator of our (1,2)-SC.

5.3 (1, n)-SC From Our (1, 2)-SC

As shown in Sec. 5.3 of [24], one can construct a 1-out-of- 2^q partial-binding commitment from a 1-out-of-2 one. We can apply the same transformation to a (1,2)-SC to get a (1, 2^q)-SC. The

communication complexity of the resulting $(1, 2^q)$ -SC is linear in q , i.e., logarithmic in the size of the committed vector.

The commitment is constructed arranging all the messages of the committed vector as the leaves of a binary tree. Each internal node is computed by computing a 1-out-of-2 commitment to its children, which are generally 1-out-of-2 commitments as well. The resulting tree has q levels. Due to the stackability properties of $(1, 2)$ -SC, we can open the commitment $(1, 2^q)$ -SC using only q openings of $(1, 2)$ -SC, that is one opening per level. Indeed at each level, the opening for one node allows to deterministically recompute all the nodes at the same level via the simulator $(1, 2)$ -SC.SimC. As pointed out in [24], to apply this transformation we need the message space and the commitment space to match. This requirement is already satisfied by our scheme, since the commitment procedure first hashes the messages to be committed. In more detail, $(1, 2^q)$ -SC works as follows.

(com, aux) $\leftarrow$ Commit($\beta, m_\beta; r$): Let $\bar{\beta}$ be the binding position β interpreted as the bit vector corresponding to the value of β (e.g., if $\beta = 2$ then $\bar{\beta} = 010$). And let $\bar{\beta}^j$ with $j \in \llbracket q \rrbracket$, the j -th bit of $\bar{\beta}$ counting from the left and starting from 0. At level j , the bit $\bar{\beta}^j$ indicates the binding position to be used in the computation of $(1, 2)$ -SC.Commit at level j . As an example consider $q = 3$, $\beta = 2$, and $\bar{\beta} = 010$. The Commit algorithm first commits to $m_\beta = m_2$ using binding position 0 as input of $(1, 2)$ -SC.Commit, then it commits to the resulting commitment using binding position 1, and finally it does the same with binding position 0. The result is the root 1-out-of-2 commitment, which is finally hashed to get **com**. For convenience, we slightly deviate from the syntax in Sec. 5.2, and Commit additionally returns a vector of auxiliary values **aux** that will be used later by the Open algorithm. In particular, **aux** contains all the commitments generated on the path from m_β to the root. However, notice that the two syntaxes are equivalent, since **aux** can be recomputed by Open itself and we use this syntax just for readability.

op $\leftarrow$ Open($\beta, \text{com}, c_{\text{com}}, \mathbf{m}, r, \mathbf{aux}$): This algorithm works starting from the leaves of the tree and progressively reaching the root. To open **com**, it starts by opening the $(1, 2)$ -SC commitment contained in **aux** $[q - 1]$. The value in **aux** $[q - 1]$ is a commitment to m_β and its sibling. This algorithm computes the opening of **aux** $[q - 1]$, that is **op** $[q - 1]$, using $(1, 2)$ -SC.Open. It then computes all the other commitments at the parent level by running $(1, 2)$ -SC.SimC($c_{\text{com}}, m_a, m_{a+1}, \mathbf{op}[q - 1]$) with $a \in \llbracket n \rrbracket$ on all consecutive pairs of messages $m_a, m_{a+1} \in \mathbf{m}$. It then iterates the same process at the upper levels, until it reaches the root and it returns a list of q openings **op**.

com $\leftarrow$ SimC($c_{\text{com}}, \mathbf{m}, \mathbf{op}$): This algorithm works starting from the leaves of the tree and progressively reaching the root. It runs $(1, 2)$ -SC.SimC on all consecutive pairs of messages in $\mathbf{m}$. It then iterates the same process at the upper levels, until it reaches the root and returns the root commitment **com**.

0/1 $\leftarrow$ ComVfy($\text{com}, c_{\text{com}}, \mathbf{op}, \mathbf{m}$): This algorithm runs SimC described above and checks if the obtained commitment is equal to **com**.

A detailed description of all the algorithms is given in Fig. 6, it uses a $(1, 2)$ -SC = (Commit', Open', SimC') as a building block.

Communication complexity. The communication complexity of our $(1, 2^q)$ -SC is $|\text{com}| + |c_{\text{com}}| + |\mathbf{op}| = l + |c_{\text{com}}| + q|\mathbf{op}_{(1,2)\text{-SC}}|$, where l is the output size of H, $|c_{\text{com}}|$ is the size of the challenge of $(1, 2)$ -SC, and $|\mathbf{op}_{(1,2)\text{-SC}}|$ is the size of the opening in $(1, 2)$ -SC. Considering our $(1, 2)$ -SC of Sec. 5.2, we have that $|c_{\text{com}}| = l$, and $|\mathbf{op}_{(1,2)\text{-SC}}| = O(\lambda) = O(\lambda^2)$. Therefore, the resulting communication complexity of $(1, 2^q)$ -SC is $O(\lambda^2 q) = O(\lambda^2 \log n)$.

$(\text{com}, \mathbf{aux}) \leftarrow \text{Commit}(\beta, m_\beta; r)$		$\text{com} \leftarrow \text{SimC}(c_{\text{com}}, \mathbf{m}, \mathbf{op})$	
1 :	parse β as a binary number $\overline{\beta}$	1 :	parse $\mathbf{op} = (\text{op}_0, \dots, \text{op}_{q-1})$
2 :	parse $r = (r_0, \dots, r_{q-1})$	2 :	$i = q - 1$, initialize empty $\mathbf{m}'$
3 :	Set $i = q - 1, v = m_\beta$	3 :	$n = \text{length}(\mathbf{m}) - 1$
4 :	Initialize empty $\mathbf{aux}$	4 :	while $i \geq 0$:
5 :	while $i \geq 0$:	5 :	$k = 0$, Initialize empty $\mathbf{m}'$
6 :	$v \leftarrow \text{Commit}'(\overline{\beta}^i, v; r_i)$	6 :	// Simulate all parent nodes
7 :	$\mathbf{aux}[i] = v$	7 :	while $k < n$:
8 :	$i = i - 1$	8 :	$u \leftarrow \text{SimC}'(c_{\text{com}}, \mathbf{m}[k], \mathbf{m}[k+1], \mathbf{op}[i])$
9 :	return $(\text{H}(v), \mathbf{aux})$	9 :	$\mathbf{m}'.\text{append}(u)$
		10 :	$k = k + 2$
		11 :	$n = \lfloor n/2 \rfloor$
		12 :	$\mathbf{m} = \mathbf{m}'$
		13 :	$i = i - 1$
		14 :	return $\text{H}(\mathbf{m}[0])$
$\mathbf{op} \leftarrow \text{Open}(\beta, \text{com}, c_{\text{com}}, \mathbf{m}, r, \mathbf{aux})$		$0/1 \leftarrow \text{ComVfy}(\text{com}, c_{\text{com}}, \mathbf{op}, \mathbf{m})$	
1 :	parse β as a binary number $\overline{\beta}$	1 :	$\text{com}' \leftarrow \text{SimC}(c_{\text{com}}, \mathbf{m}, \mathbf{op})$
2 :	parse $r = (r_0, \dots, r_{q-1})$	2 :	if $\text{com} = \text{com}'$:
3 :	$n = \text{length}(\mathbf{m}) - 1$	3 :	return 1
4 :	$i = q - 1$, Initialize empty $\mathbf{op}$	4 :	else :
5 :	while $i \geq 0$	5 :	return 0
6 :	// Open the binding position and its sibling		
7 :	if $\overline{\beta}^i = 0$:		
8 :	$v_0 = \mathbf{m}[\beta], v_1 = \mathbf{m}[\beta + 1], j = \beta$		
9 :	else		
10 :	$v_0 = \mathbf{m}[\beta - 1], v_1 = \mathbf{m}[\beta], j = \beta - 1$		
11 :	$\mathbf{op}[i] \leftarrow \text{Open}'(\overline{\beta}^i, \mathbf{aux}[i], c_{\text{com}}, v_0, v_1, r_i)$		
12 :	// Run the simulator to get the parent nodes		
13 :	$k = 0$, Initialize empty $\mathbf{m}'$		
14 :	while $k < n$:		
15 :	$u \leftarrow \text{SimC}'(c_{\text{com}}, \mathbf{m}[k], \mathbf{m}[k+1], \mathbf{op}[i])$		
16 :	$\mathbf{m}'.\text{append}(u)$		
17 :	$k = k + 2$		
18 :	$n = \lfloor n/2 \rfloor, \beta = \lfloor \beta/2 \rfloor, i = i - 1$		
19 :	$\mathbf{m} = \mathbf{m}'$ // Messages of the next iteration		
20 :	return $\mathbf{op}$		

Fig. 6: $(1, 2^q)$ -SC from $(1, 2)$ -SC.

Theorem 3. *If $(1, 2)$ -SC is the 1-out-of-2 stackable commitment of Sec. 5.2, then $(1, 2^q)$ -SC described in Fig. 6 is a 1-out-of- 2^q stackable commitment (non-programmable) ROM.*

Proof. Completeness follows by inspection. We prove all the other properties below.

Recyclable Openings: The distribution $D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ of $(1, 2^q)$ -SC is simply constructed by sampling q openings from the distribution $D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ of $(1, 2)$ -SC. Indistinguishability from openings of a regular protocol run can be argued as in the proof of the recyclable opening property in Sec. 5.2.

Extended Honest-Receiver Hiding: It simply follows from the EHRH of $(1, 2)$ -SC by simply observing that q nested runs of $(1, 2)$ -SC are statistically indistinguishable from q nested runs of $(1, 2)$ -SC.SimC.

1-Binding: It follows from the 1-Binding of $(1, 2)$ -SC. In particular, given $\mathcal{A}^{(1, 2^q)\text{-SC}} = (\mathcal{A}_0^{(1, 2^q)\text{-SC}}, \mathcal{A}_1^{(1, 2^q)\text{-SC}})$ which wins the 1-Binding experiment w.r.t. $(1, 2^q)$ -SC with non-negligible probability, we build $\mathcal{A}^{(1, 2)\text{-SC}} = (\mathcal{A}_0^{(1, 2)\text{-SC}}, \mathcal{A}_1^{(1, 2)\text{-SC}})$ breaking the 1-Binding of $(1, 2)$ -SC with related non-negligible probability in the following way.

$\mathcal{A}_0^{(1, 2)\text{-SC}}$ activates $\mathcal{A}_0^{(1, 2^q)\text{-SC}}$ and does the following steps:

1. Upon receiving a query with inp_H made by $\mathcal{A}_0^{(1, 2^q)\text{-SC}}$, query H obtain out_H , send it to $\mathcal{A}_0^{(1, 2^q)\text{-SC}}$, and record $(\text{inp}_H, \text{out}_H)$ in the list $\mathcal{L}_Q$.
2. Upon receiving $(\text{com}^{(1, 2^q)\text{-SC}}, \text{st}^{(1, 2^q)\text{-SC}})$ from $\mathcal{A}_0^{(1, 2^q)\text{-SC}}$, chose a random element $(\text{inp}_H^*, \text{out}_H^*)$ of $\mathcal{L}_Q$ and set inp_H^* as a commitment $\text{com}^{(1, 2)\text{-SC}}$.
3. Output $\text{com}^{(1, 2)\text{-SC}}, \text{st}^{(1, 2)\text{-SC}} = (\text{st}^{(1, 2^q)\text{-SC}})$.

$\mathcal{A}_1^{(1, 2)\text{-SC}}$ on input the random challenges $c_{\text{com}}^0, c_{\text{com}}^1$ and the output $(\text{com}^{(1, 2)\text{-SC}}, \text{st}^{(1, 2)\text{-SC}})$ $\mathcal{A}_0^{(1, 2)\text{-SC}}$ activates $\mathcal{A}_1^{(1, 2^q)\text{-SC}}$ on input $(\text{st}^{(1, 2^q)\text{-SC}}, c_{\text{com}}^0, c_{\text{com}}^1)$ and does the following steps:

1. Upon receiving $(\text{op}^0, \text{op}^1, \mathbf{m}^0, \mathbf{m}^1)$ from $\mathcal{A}_1^{(1, 2^q)\text{-SC}}$ reconstruct the tree T^b using $(1, 2^q)$ -SC.SimC($c_{\text{com}}^b, \mathbf{m}^b, \text{op}^b$) with $b \in \{0, 1\}$.
2. Parse T^b from the root until a node that is equal to $\text{com}^{(1, 2)\text{-SC}}$ is found, with $b \in \{0, 1\}$. If such node is not found, output $\perp$.
3. Output the children nodes $\widehat{\mathbf{m}}^0$ in the first tree T^0 and the children nodes $\widehat{\mathbf{m}}^1$ in the second tree T^1 , along with their openings $\widehat{\text{op}}^0$ and $\widehat{\text{op}}^1$ respectively.

We now argue that $\mathcal{A}^{(1, 2)\text{-SC}}$ wins the 1-Binding game at least with the same probability with which $\mathcal{A}^{(1, 2^q)\text{-SC}}$ wins the 1-Binding game divided by Q_H (minus a negligible factor), where $Q_H = |\mathcal{L}_Q|$.

Observation: Since $\mathcal{A}^{(1, 2^q)\text{-SC}}$ wins the 1-Binding game with non-negligible probability, it holds by inspection that in the two trees T^0 and T^1 there is a path going from the root $\text{com}^{(1, 2^q)\text{-SC}}$ down to a node $\widehat{\text{com}}$ s.t. (i) $\widehat{\text{com}}$ is present in both trees⁹ with children $\widehat{\mathbf{m}}^0$ and $\widehat{\mathbf{m}}^1$ respectively, and (ii) these children are such that $\widehat{\mathbf{m}}^0[0] \neq \widehat{\mathbf{m}}^1[0]$ and $\widehat{\mathbf{m}}^0[0] \neq \widehat{\mathbf{m}}^1[0]$.

We now argue that $\widehat{\text{com}}$ must be, except with negligible probability, in the queries made by $\mathcal{A}_0^{(1, 2^q)\text{-SC}}$. Note that the openings output by $\mathcal{A}_1^{(1, 2^q)\text{-SC}}$ are accepting, therefore $H(\widehat{\text{com}})$ must be coherent with the hash chain that goes from $\widehat{\text{com}}$ up until the root $\text{com}^{(1, 2^q)\text{-SC}}$. Since the budget of queries to the random oracle of $\mathcal{A}^{(1, 2^q)\text{-SC}}$ is polynomial, the probability that $\mathcal{A}_1^{(1, 2^q)\text{-SC}}$ found a compatible $\widehat{\text{com}}$ that was not queried by $\mathcal{A}_0^{(1, 2^q)\text{-SC}}$ is negligible.

⁹ Notice that this includes the case where the root $\text{com}^{(1, 2^q)\text{-SC}}$ is equal to $\widehat{\text{com}}$.

To complete the proof, it remains to observe now that the probability that, given that $\widehat{\text{com}}$ is in the queries made by $\mathcal{A}_0^{(1,2^q)\text{-SC}}$, the randomly selected $\text{com}^{(1,2)\text{-SC}}$ is equal to $\widehat{\text{com}}$ is at least $1/Q_H$.

6 Our Disjunctive Composition

Given a stackable Σ -Protocol for language $\mathcal{L}$, $\Sigma = (P_0, P_1, V)$, and a 1-out-of- n stackable commitment $(1, n)\text{-SC} = (\text{Setup}, \text{Commit}, \text{Open}, \text{ComVfy})$, we define in Fig. 7 an EHVZK 3-round PoK for language $\mathcal{L}_{\text{OR}} = \{x_0, \dots, x_{n-1} \mid \exists i \in \llbracket n \rrbracket : x_i \in \mathcal{L}\}$. Our composition works in the same way as [25], but we plug in our weaker tool namely $(1, n)\text{-SC}$ and we prove that this approach leads to an EHVZK 3-round PoK for $\mathcal{L}_{\text{OR}}$. In more details, the composition works as follows.

Round 1: In the first round, the prover uses its witness w_i for the statement x_i to run the prover algorithm of the base Σ protocol P_0 and compute the corresponding first-round message a_i . The prover then commits to a_i using $(1, n)\text{-SC}$ and sends the resulting commitment com to the verifier.

Round 2: The verifier sends a uniformly random challenge $c = (c_\Sigma, c_{\text{com}})$.

Round 3: The prover runs $\Sigma.P_1$ on input $(x_i, w_i, a_i, c_\Sigma)$ which returns the corresponding third-round message z^* . The prover then runs the simulator of Σ on all the instances $x_j, j \in \llbracket n \rrbracket, j \neq i$ to get the first-round messages a_j . It then opens the commitment to $(a_0, \dots, a_{n-1})$ and sends the opening op and z^* to the verifier.

Verification: The verifier recomputes $(a_0, \dots, a_{n-1})$ running the simulator, it verifies that each (a_i, c_Σ, z^*) with $i \in \llbracket n \rrbracket$ is an accepting transcript for x_i , and it verifies the opening op w.r.t the values $(a_0, \dots, a_{n-1})$. If all the checks are successful it outputs 1, otherwise it outputs 0.

Communication complexity. The communication complexity of the protocol in Fig. 7 instantiated using $(1, n)\text{-SC}$ from Sec. 5.2 is $|\text{com}| + |\text{op}| + |c| + |z| = O(\lambda^2 \log n) + |c_\Sigma| + |z| = O(\lambda^2 \log n) + \text{CC}(\Sigma)$, where $\text{CC}(\Sigma)$ is the communication complexity of the base protocol Σ .

Theorem 4. *Let $\Sigma = (P_0, P_1, V)$ be a stackable Σ -Protocol for language $\mathcal{L}$, and let $(1, n)\text{-SC} = (\text{Setup}, \text{Commit}, \text{Open}, \text{ComVfy})$ be a 1-out-of- n stackable commitment, then the protocol of Fig. 7 is a EHVZK 3-round PoK for $\mathcal{L}_{\text{OR}}$.*

Proof. We prove first that Fig. 7 enjoys (i) completeness, (ii) EHVZK with recyclable third messages. These proofs are almost identical to the ones in [25] therefore, we just report a proof sketch below.

Completeness. It follows directly from the completeness of Σ and of $(1, n)\text{-SC}$.

Statistical EHVZK and Recyclable Third Messages.

For $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ we define the third-round message distribution as $D_{\text{pp}, c_\Sigma}^{(z)} = \{z = (z^*, \text{op}) \mid z^* \leftarrow \$ D_{\text{pp}, c_\Sigma}^{(z)}, \text{op} \leftarrow \$ D_{\text{pp}, c_{\text{com}}}^{(\text{op})}\}$, and the simulator Sim^{OR} below.

```

SimOR(( $x_0, \dots, x_{n-1}$ ),  $c$ , ( $z^*, \text{op}$ ))
1 : parse  $c = (c_\Sigma, c_{\text{com}})$ 
2 : for  $i \in \llbracket n \rrbracket$  :  $a_i \leftarrow \text{Sim}_\Sigma(x_i, c_\Sigma, z^*)$ 
3 : Set  $\mathbf{a} = (a_0, \dots, a_{n-1})$ 
4 :  $\text{com} \leftarrow \text{SimC}(c_{\text{com}}, \mathbf{a}, \text{op})$ 
5 : return  $\text{com}$ 

```


$P_0^{\text{OR}}((x_0, \dots, x_{n-1}), (w, i); r)$	$V^{\text{OR}}((x_0, \dots, x_{n-1}), a, c, z)$
1 : parse $r = (r_0, r_1)$	1 : parse $z = (z^*, \text{op})$
2 : $a_i \leftarrow P_0(x_i, w; r_0)$	2 : parse $c = (c_\Sigma, c_{\text{com}})$
3 : $\text{com} \leftarrow \text{Commit}(i, a_i; r_1)$	3 : parse $a = \text{com}$
4 : return $a = \text{com}$	4 : $\forall i \in \llbracket n \rrbracket : a_i \leftarrow \text{Sim}_\Sigma(x_i, c_\Sigma, z^*)$
$P_1^{\text{OR}}((x_0, \dots, x_{n-1}), (w, i), a, c, r)$	5 : Set $\mathbf{a} = (a_0, \dots, a_{n-1})$
1 : parse $r = (r_0, r_1)$	6 : if $\text{ComVfy}(\text{com}, c_{\text{com}}, \text{op}, \mathbf{a}) = 0$
2 : parse $c = (c_\Sigma, c_{\text{com}})$	7 : return 0
3 : parse $a = \text{com}$	8 : if $\exists j \in \llbracket n \rrbracket : V(x_j, a_j, c_\Sigma, z^*) = 0$
4 : $a_i \leftarrow P_0(x_i, w; r_0)$	9 : return 0
5 : $z_i \leftarrow P_1(x_i, w, a_i, c_\Sigma, r_0)$	10 : return 1
6 : $\forall j \in \llbracket n \rrbracket, j \neq i : a_j \leftarrow \text{Sim}_\Sigma(x_j, c_\Sigma, z_i)$	Setup (1^λ)
7 : Set $\mathbf{a} = (a_0, \dots, a_{n-1})$	1 : $\text{pp}_\Sigma \leftarrow \Sigma.\text{Setup}(1^\lambda)$
8 : $\text{op} \leftarrow \text{Open}(i, \text{com}, c_{\text{com}}, \mathbf{a}, r_1)$	2 : $\text{pp}_{\text{com}} \leftarrow (1, n)\text{-SC}.\text{Setup}(1^\lambda)$
9 : return $z = (z_i, \text{op})$	3 : return $\text{pp} = (\text{pp}_\Sigma, \text{pp}_{\text{com}})$

Fig. 7: EHVZK 3-round PoK for $\mathcal{L}_{\text{OR}}$. A setup is not required if one uses our $(1, n)$ -SC of Sec. 5 and a base stackable Σ -protocol without a setup.

We show that sampling $z = (\text{com}, z^*)$ from $D_{\text{pp}, c_\Sigma}^{(z)}$ and running the simulator above is statistically indistinguishable from the distribution of a regular execution of Σ^{OR} via the sequence of indistinguishable hybrids below:

$\mathcal{H}_0$: It is the distribution of a regular execution of Σ .

$\mathcal{H}_1$: It is identical to $\mathcal{H}_0$ except that the first-round message a_i of the active clause x_i is generated by sampling $z^* \leftarrow D_{\text{pp}, c_\Sigma}^{(z)}$ and running $\text{Sim}_\Sigma(x_i, c_\Sigma, z^*)$ instead of $P_0(x_i, w)$.

$\mathcal{H}_2$: It is identical to $\mathcal{H}_1$ except that com is generated by sampling $\text{op} \leftarrow D_{\text{pp}, c_{\text{com}}}^{(\text{op})}$ and running $\text{com} \leftarrow \text{SimC}(c_{\text{com}}, \mathbf{a}, \text{op})$ instead of $\text{Commit}(i, a_i)$.

The proof is concluded by observing that $\mathcal{H}_0 \approx_s \mathcal{H}_1$ thanks to EHVZK with recyclable third messages of Σ , that $\mathcal{H}_1 \approx_s \mathcal{H}_2$ thanks to EHRH with recyclable openings of $(1, n)$ -SC, and that $\mathcal{H}_2$ coincides with Sim^{OR} .

Proof of Knowledge. To complete proof of Theorem 4 we need to prove that $\Sigma^{\text{OR}} = (P_0^{\text{OR}}, P_1^{\text{OR}}, V^{\text{OR}})$ is a proof of knowledge. We give a high-level description of our extractor. A detailed description and the analysis of its expected running time and its failure probability are given in App. A.

High-level description of our extractor Ext_{OR} . Our extractor Ext_{OR} has oracle access to a malicious prover $\hat{P}^{\text{OR}}$ that proves statement $(x_0, \dots, x_{n-1})$. Ext_{OR} uses $\hat{P}^{\text{OR}}$ to build n provers P_Σ^i of n different instances of the Σ -protocols. Since each Σ -protocol instance is also a PoK, it admits an extractor Ext_Σ^i , for $i \in \llbracket n \rrbracket$. Therefore, our extractor Ext_{OR} runs Ext_Σ^i , impersonating for him the prover P_Σ^i of the Σ -protocol, for all $i \in \llbracket n \rrbracket$.

In more detail, let us focus on how Ext_{OR} emulates prover P_Σ^i . Ext_{OR} obtains the transcript $(\text{com}, c = (c_\Sigma, c_{\text{com}}), z = (z^*, \text{op}))$ from $\hat{P}^{\text{OR}}$, computes a_i from (z^*, op) as V^{OR} would do, and

forwards a_i to Ext_Σ^i . Then, once Ext_{OR} gets the challenge c'_Σ from Ext_Σ^i , Ext_{OR} rewinds $\hat{\text{P}}^{\text{OR}}$ feeding it the challenge $c' = (c'_\Sigma, c'_{\text{com}})$ for uniformly random c'_{com} . Ext_{OR} obtains $(\text{com}, c' = (c'_\Sigma, c'_{\text{com}}), \hat{z} = (\hat{z}^*, \hat{\text{op}}))$ from $\hat{\text{P}}^{\text{OR}}$. At this point, note that the first rounds of the i -th execution of the Σ -protocol (namely $\hat{a}_i$ obtained from $\hat{z}$) could be different from the first round of the i -th execution of the Σ -protocol (namely a_i) before the rewind. If this is the case, the i -th execution is aborted, otherwise Ext_{OR} , acting as P_Σ^i , forwards $\hat{z}^*$ to Ext_Σ^i .

It follows from the 1-binding property of $(1, n)$ -SC that at least for one of n executions, let us say the j -th execution, the event described above does not occur, therefore in this case Ext_{OR} perfectly emulates the malicious prover P_Σ^j . From the PoK property of Σ -protocols, it follows that the extractor Ext_Σ^j is extracting the witness w from malicious prover P_Σ^j w.r.t. instance x_j . Once this point is reached Ext_{OR} outputs w .

The final observation is that we consider the extractor of Σ -protocols that interacts with the prover once and outputs fail if it does not get an accepting transcript (e.g, see Theorem 7.1 of [21]). Then, the extractor fixes the first round-message of the prover and queries it with fresh challenges until a second accepting transcript is gathered. If too many rewinding attempts (i.e., 2^λ) are performed the extractor outputs fail and terminates. We observe that we can run n instances of such extractor in parallel (one per instance in the disjunction), providing them the same random coins. In this way, they will all have the same behavior. That is (1) At the i -th rewinding step, they all send the same challenge c_Σ^i (2) If they do not get an accepting transcript, they unanimously decide to make another rewinding attempt or stop (3) They all output a witness after a second valid transcript.

Our NIZK PoK for Disjunctions. Applying the Fiat-Shamir transform to the protocol of Fig. 7 yields a NIZK PoK in the ROM. The statistical ZK follows from the statistical EHVZK of OR-composition in Fig. 7. The PoK property follow from the one of Fig. 7 using standard results [42].

6.1 Concluding Remarks

Several extensions to our work are possible, such as considering instances belonging to different languages in the disjunction, or proving validity of at least k out of n instances or a combination of the two. The case of multiple language is simply dealt with by our solution plugging our commitment in the cross-stacking compiler of [25]. The case of k -out-of- n proofs is more complicated instead. While the techniques in [7] and [25] deal with k -out-of- n proofs starting from the disjunctive compiler of [25], they either rely on number-theoretic assumptions or their adaptation to our commitment scheme would most likely require to heuristically assume that it is secure when H is a collision-resistant hash function. The fact that previous techniques for k -out-of- n proofs do not generalize to our case, although they are also built starting from the stacking compiler of [25], is an indication that our techniques deviate from the previous ones and obtaining compactness and statistical ZK without hardness assumptions appears to be challenging.

7 Acknowledgments

This work is supported by the PICOCRYPT project that has received funding from the European Research Council (ERC) under the European Unions Horizon 2020 research and innovation programme (Grant agreement No. 101001283), partially supported by projects PRODIGY (TED2021-132464B-I00) and ESPADA (PID2022-142290OB-I00) funded by MCIN/AEI/10.13039/501100011033/.

This work is also part of the grants CEX2024-001471-M funded by MICIU/AEI/10.13039/501100011033, and JDC2023-050791-I funded by MCIN/AEI/10.13039/501100011033 and the ESF+. The last author is member of the Gruppo Nazionale Calcolo Scientifico-Istituto Nazionale di Alta Matematica (GNCS-INdAM) and his work was partially supported by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

References

1. Abe, M., Balbás, D., Bui, D., Ohkubo, M., Shang, Z., Tibouchi, M.: Critical round in multi-round proofs: Compositions and transformation to trapdoor commitments. Cryptology ePrint Archive, Paper 2024/1766 (2024), <https://eprint.iacr.org/2024/1766>
2. Abe, M., Bogdanov, A., Ohkubo, M., Rosen, A., Shang, Z., Tibouchi, M.: CDS composition of multi-round protocols. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part IX. LNCS, vol. 14928, pp. 391–423. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68400-5_12
3. Attema, T., Cramer, R.: Compressed Σ -protocol theory and practical application to plug & play secure algorithmics. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part III. LNCS, vol. 12172, pp. 513–543. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2020). https://doi.org/10.1007/978-3-030-56877-1_18
4. Attema, T., Cramer, R., Fehr, S.: Compressing proofs of k-out-of-n partial knowledge. In: Malkin, T., Peikert, C. (eds.) CRYPTO 2021, Part IV. LNCS, vol. 12828, pp. 65–91. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84259-8_3
5. Attema, T., Cramer, R., Kohl, L.: A compressed Σ -protocol theory for lattices. In: Malkin, T., Peikert, C. (eds.) CRYPTO 2021, Part II. LNCS, vol. 12826, pp. 549–579. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84245-1_19
6. Avitabile, G., Botta, V., Friolo, D., Venturi, D., Visconti, I.: Compact proofs of partial knowledge for overlapping CNF formulae. J. Cryptol. **38**(1), 7 (2025). <https://doi.org/10.1007/S00145-024-09532-3>, <https://doi.org/10.1007/s00145-024-09532-3>
7. Avitabile, G., Botta, V., Friolo, D., Visconti, I.: Efficient proofs of knowledge for threshold relations. In: Atluri, V., Di Pietro, R., Jensen, C.D., Meng, W. (eds.) ESORICS 2022, Part III. LNCS, vol. 13556, pp. 42–62. Springer, Cham, Switzerland, Copenhagen, Denmark (Sep 26–30, 2022). https://doi.org/10.1007/978-3-031-17143-7_3
8. Bellare, M., Ristov, T.: Hash functions from sigma protocols and improvements to VSH. In: Pieprzyk, J. (ed.) ASIACRYPT 2008. LNCS, vol. 5350, pp. 125–142. Springer Berlin Heidelberg, Germany, Melbourne, Australia (Dec 7–11, 2008). https://doi.org/10.1007/978-3-540-89255-7_9
9. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive oracle proofs. In: Hirt, M., Smith, A.D. (eds.) TCC 2016-B, Part II. LNCS, vol. 9986, pp. 31–60. Springer Berlin Heidelberg, Germany, Beijing, China (Oct 31 – Nov 3, 2016). https://doi.org/10.1007/978-3-662-53644-5_2
10. Blum, M.: How to prove a theorem so no one else can claim it. In: International Congress of Mathematicians (August 1986), invited 45 minute address
11. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 327–357. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). https://doi.org/10.1007/978-3-662-49896-5_12
12. Boudgoust, K., Simkin, M.: The power of NAPs: - compressing OR-proofs via collision-resistant hashing. In: Boyle, E., Mahmoody, M. (eds.) TCC 2024, Part I. LNCS, vol. 15364, pp. 35–66. Springer, Cham, Switzerland, Milan, Italy (Dec 2–6, 2024). https://doi.org/10.1007/978-3-031-78011-0_2
13. Boudgoust, K., Simkin, M.: The power of NAPs: Compressing OR-proofs via collision-resistant hashing. Cryptology ePrint Archive, Paper 2024/1482 (2024), <https://eprint.iacr.org/2024/1482>
14. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE Symposium on Security and Privacy. pp. 315–334. IEEE Computer Society Press, San Francisco, CA, USA (May 21–23, 2018). <https://doi.org/10.1109/SP.2018.00020>
15. Chaum, D., Pedersen, T.P.: Transferred cash grows in size. In: Rueppel, R.A. (ed.) EUROCRYPT’92. LNCS, vol. 658, pp. 390–407. Springer Berlin Heidelberg, Germany, Balatonfüred, Hungary (May 24–28, 1993). https://doi.org/10.1007/3-540-47555-9_32

16. Ciampi, M., Persiano, G., Scafuro, A., Siniscalchi, L., Visconti, I.: Improved OR-composition of sigma-protocols. In: Kushilevitz, E., Malkin, T. (eds.) TCC 2016-A, Part II. LNCS, vol. 9563, pp. 112–141. Springer Berlin Heidelberg, Germany, Tel Aviv, Israel (Jan 10–13, 2016). https://doi.org/10.1007/978-3-662-49099-0_5
17. Ciampi, M., Persiano, G., Scafuro, A., Siniscalchi, L., Visconti, I.: Online/offline OR composition of sigma protocols. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 63–92. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). https://doi.org/10.1007/978-3-662-49896-5_3
18. Cramer, R., Damgård, I.: Zero-knowledge proofs for finite field arithmetic; or: Can zero-knowledge be for free? In: Krawczyk, H. (ed.) CRYPTO'98. LNCS, vol. 1462, pp. 424–441. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 23–27, 1998). <https://doi.org/10.1007/BFb0055745>
19. Cramer, R., Damgård, I., Schoenmakers, B.: Proofs of partial knowledge and simplified design of witness hiding protocols. In: Desmedt, Y. (ed.) CRYPTO'94. LNCS, vol. 839, pp. 174–187. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 21–25, 1994). https://doi.org/10.1007/3-540-48658-5_19
20. Dutta, M., Ganesh, C., Jawalkar, N.: Succinct verification of compressed sigma protocols in the updatable SRS setting. In: Tang, Q., Teague, V. (eds.) PKC 2024, Part II. LNCS, vol. 14602, pp. 305–336. Springer, Cham, Switzerland, Sydney, NSW, Australia (Apr 15–17, 2024). https://doi.org/10.1007/978-3-031-57722-2_10
21. Feige, U., Shamir, A.: Zero knowledge proofs of knowledge in two rounds. In: Brassard, G. (ed.) CRYPTO'89. LNCS, vol. 435, pp. 526–544. Springer, New York, USA, Santa Barbara, CA, USA (Aug 20–24, 1990). https://doi.org/10.1007/0-387-34805-0_46
22. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over lagrange-bases for oecumenical non-interactive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953 (2019), <https://eprint.iacr.org/2019/953>
23. Gentry, C., Wichs, D.: Separating succinct non-interactive arguments from all falsifiable assumptions. In: Fortnow, L., Vadhan, S.P. (eds.) 43rd ACM STOC. pp. 99–108. ACM Press, San Jose, CA, USA (Jun 6–8, 2011). <https://doi.org/10.1145/1993636.1993651>
24. Goel, A., Green, M., Hall-Andersen, M., Kaptchuk, G.: Stacking sigmas: A framework to compose σ -protocols for disjunctions. Cryptology ePrint Archive, Paper 2021/422 (2021), <https://eprint.iacr.org/2021/422>
25. Goel, A., Green, M., Hall-Andersen, M., Kaptchuk, G.: Stacking sigmas: A framework to compose Σ -protocols for disjunctions. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part II. LNCS, vol. 13276, pp. 458–487. Springer, Cham, Switzerland, Trondheim, Norway (May 30 – Jun 3, 2022). https://doi.org/10.1007/978-3-031-07085-3_16
26. Goel, A., Hall-Andersen, M., Kaptchuk, G., Spooner, N.: Speed-stacking: Fast sublinear zero-knowledge proofs for disjunctions. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part II. LNCS, vol. 14005, pp. 347–378. Springer, Cham, Switzerland, Lyon, France (Apr 23–27, 2023). https://doi.org/10.1007/978-3-031-30617-4_12
27. Goldreich, O., Micali, S., Wigderson, A.: Proofs that yield nothing but their validity and a methodology of cryptographic protocol design (extended abstract). In: 27th FOCS. pp. 174–187. IEEE Computer Society Press, Toronto, Ontario, Canada (Oct 27–29, 1986). <https://doi.org/10.1109/SFCS.1986.47>
28. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 305–326. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). https://doi.org/10.1007/978-3-662-49896-5_11
29. Groth, J., Kohlweiss, M.: One-out-of-many proofs: Or how to leak a secret and spend a coin. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part II. LNCS, vol. 9057, pp. 253–280. Springer Berlin Heidelberg, Germany, Sofia, Bulgaria (Apr 26–30, 2015). https://doi.org/10.1007/978-3-662-46803-6_9
30. Guillou, L.C., Quisquater, J.J.: A “paradoxical” indentivity-based signature scheme resulting from zero-knowledge. In: Goldwasser, S. (ed.) CRYPTO'88. LNCS, vol. 403, pp. 216–231. Springer, New York, USA, Santa Barbara, CA, USA (Aug 21–25, 1990). https://doi.org/10.1007/0-387-34799-2_16
31. Hazay, C., Venkatasubramanian, M.: On the power of secure two-party computation. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part II. LNCS, vol. 9815, pp. 397–429. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 14–18, 2016). https://doi.org/10.1007/978-3-662-53008-5_14
32. Heath, D., Kolesnikov, V.: Stacked garbling for disjunctive zero-knowledge proofs. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part III. LNCS, vol. 12107, pp. 569–598. Springer, Cham, Switzerland, Zagreb, Croatia (May 10–14, 2020). https://doi.org/10.1007/978-3-030-45727-3_19
33. Heath, D., Kolesnikov, V., Peceny, S.: Garbling, stacked and staggered - faster k-out-of-n garbled function evaluation. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part II. LNCS, vol. 13091, pp. 245–274. Springer, Cham, Switzerland, Singapore (Dec 6–10, 2021). https://doi.org/10.1007/978-3-030-92075-3_9
34. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge from secure multiparty computation. In: Johnson, D.S., Feige, U. (eds.) 39th ACM STOC. pp. 21–30. ACM Press, San Diego, CA, USA (Jun 11–13, 2007). <https://doi.org/10.1145/1250790.1250794>

35. Khurana, D., Ostrovsky, R., Srinivasan, A.: Round optimal black-box “commit-and-prove”. In: Beimel, A., Dziembowski, S. (eds.) TCC 2018, Part I. LNCS, vol. 11239, pp. 286–313. Springer, Cham, Switzerland, Panaji, India (Nov 11–14, 2018). https://doi.org/10.1007/978-3-030-03807-6_11
36. Kilian, J.: A note on efficient zero-knowledge proofs and arguments (extended abstract). In: 24th ACM STOC. pp. 723–732. ACM Press, Victoria, BC, Canada (May 4–6, 1992). <https://doi.org/10.1145/129712.129782>
37. Maurer, U.: Zero-knowledge proofs of knowledge for group homomorphisms. Des. Codes Cryptogr. **77**(2-3), 663–676 (2015). <https://doi.org/10.1007/S10623-015-0103-5>, <https://doi.org/10.1007/s10623-015-0103-5>
38. Micali, S.: Computationally sound proofs. SIAM J. Comput. **30**(4), 1253–1298 (2000). <https://doi.org/10.1137/S0097539795284959>, <https://doi.org/10.1137/S0097539795284959>
39. Ostrovsky, R., Richelson, S., Scafuro, A.: Round-optimal black-box two-party computation. In: Gennaro, R., Robshaw, M.J.B. (eds.) CRYPTO 2015, Part II. LNCS, vol. 9216, pp. 339–358. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 16–20, 2015). https://doi.org/10.1007/978-3-662-48000-7_17
40. Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: Nearly practical verifiable computation. In: 2013 IEEE Symposium on Security and Privacy. pp. 238–252. IEEE Computer Society Press, Berkeley, CA, USA (May 19–22, 2013). <https://doi.org/10.1109/SP.2013.47>
41. Pass, R., Rosen, A.: New and improved constructions of non-malleable cryptographic protocols. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC. pp. 533–542. ACM Press, Baltimore, MA, USA (May 22–24, 2005). <https://doi.org/10.1145/1060590.1060670>
42. Pointcheval, D., Stern, J.: Security proofs for signature schemes. In: Maurer, U.M. (ed.) EUROCRYPT’96. LNCS, vol. 1070, pp. 387–398. Springer Berlin Heidelberg, Germany, Saragossa, Spain (May 12–16, 1996). https://doi.org/10.1007/3-540-68339-9_33
43. Reingold, O., Rothblum, G.N., Rothblum, R.D.: Constant-round interactive proofs for delegating computation. In: Wichs, D., Mansour, Y. (eds.) 48th ACM STOC. pp. 49–62. ACM Press, Cambridge, MA, USA (Jun 18–21, 2016). <https://doi.org/10.1145/2897518.2897652>
44. Schnorr, C.P.: Efficient identification and signatures for smart cards. In: Brassard, G. (ed.) CRYPTO’89. LNCS, vol. 435, pp. 239–252. Springer, New York, USA, Santa Barbara, CA, USA (Aug 20–24, 1990). https://doi.org/10.1007/0-387-34805-0_22
45. Waters, B., Wu, D.J.: Batch arguments for NP and more from standard bilinear group assumptions. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part II. LNCS, vol. 13508, pp. 433–463. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15979-4_15
46. Yang, Y., Heath, D., Hazay, C., Kolesnikov, V., Venkatasubramanian, M.: Batchman and robin: Batched and non-batched branching for interactive ZK. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 1452–1466. ACM Press, Copenhagen, Denmark (Nov 26–30, 2023). <https://doi.org/10.1145/3576915.3623169>

Appendix

A Detailed Security Proof for Our Knowledge Extractor

Full description of our extractor Ext_{OR} . Our extractor Ext_{OR} operates in two phases:

- (1) **Get an accepting transcript of Σ^{OR} :** Ext_{OR} is given oracle access to a malicious prover $\hat{\text{P}}^{\text{OR}}$ that proves statement $(x_0, \dots, x_{n-1})$. Using black-box access to $\hat{\text{P}}^{\text{OR}}$, Ext_{OR} internally simulates the role of the honest verifier V^{OR} (i.e., it sends uniformly random $c = (c_\Sigma, c_{\text{com}})$) for $\hat{\text{P}}^{\text{OR}}$ and outputs fail if $\hat{\text{P}}^{\text{OR}}$ does not produce an accepting proof. Let $(\text{com}, c = (c_\Sigma, c_{\text{com}}), z = (z^*, \text{op}))$ be the transcript resulting from a successful emulation by Ext_{OR} .
- (2) **Use the extractor of the base Σ -Protocol:** Ext_{OR} obtains $a_0, \dots, a_{n-1}$ from (z^*, op) as V^{OR} would do and activates $\llbracket n \rrbracket$ executions of Ext_Σ^i with the same random tape, one for each x_i with $i \in \llbracket n \rrbracket$. The extractor Ext_{OR} initializes the set $\text{alive} = \llbracket n \rrbracket$ and acts as a prover P_Σ^i of Σ with each Ext_Σ^i forwarding a_i . Then Ext_{OR} computes the following steps:
 1. Upon receiving c'_Σ from Ext_Σ^i (note that c'_Σ is the same for all Ext_Σ^i , for all $i \in \text{alive}$, since they are executed with the same random coins). Feed $c = (c'_\Sigma, c'_{\text{com}})$, with uniformly sampled c'_{com} , to $\hat{\text{P}}^{\text{OR}}$ and do the following:
 - (a) If $\hat{\text{P}}^{\text{OR}}$ replies with $\hat{z} = (\hat{z}^*, \hat{\text{op}})$ and $\text{V}^{\text{OR}}((x_0, \dots, x_{n-1}), \text{com}, c'_{\text{com}}, (\hat{z}^*, \hat{\text{op}})) = 1$, do the following:
 - i. Obtain $\hat{a}_0, \dots, \hat{a}_{n-1}$ from $(\hat{z}^*, \hat{\text{op}})$ as V^{OR} .
 - ii. Let cnt be the set of indices for which $\hat{a}_i = a_i$, for $i \in \text{alive}$, set $\text{alive} = \text{cnt}$.
 - iii. If $\text{alive} = \emptyset$, exit the extraction procedure and output fail.
 - iv. Forward $\hat{z}^*$ to Ext_Σ^i for $i \in \text{alive}$.
 - (b) If $\hat{\text{P}}^{\text{OR}}$ does not reply or $\text{V}^{\text{OR}}((x_0, \dots, x_{n-1}), \text{com}, c'_{\text{com}}, (\hat{z}^*, \hat{\text{op}})) = 0$, send $\perp$ to Ext_Σ^i for $i \in \text{alive}$.
 2. Ext_{OR} takes a decision depending on the output of the extractors Ext_Σ^i for $i \in \text{alive}$, that is of the same type for all of them due to their deterministic (once the random coins are fixed) extraction strategy:
 - (a) If the extractors Ext_Σ^i for $i \in \text{alive}$ output a witness, look for the first witness w_i with $i \in [\text{alive}]$ so that $(x_i, w_i) \in R$ and output (w_i, i) . If there is no such an index i , output fail.
 - (b) If the extractors Ext_Σ^i for $i \in \text{alive}$ supply a new challenge c'_Σ , go back to Step (1).
 - (c) If the extractors Ext_Σ^i for $i \in \text{alive}$ output fail, output fail.

It remains to show that Ext_{OR} satisfies Definition 4

Analysis of the running time of Ext_{OR} . Let p be the conditional probability of $\hat{\text{P}}^{\text{OR}}$ completing the protocol after it sent the first-round message. Then the expected running time of Ext_{OR} is $(1 - p) \cdot 1 + p \cdot 1/p < 2$ times the running time of V^{OR} , which is polynomial.

Analysis of the probability of failure of Ext_{OR} . It remains to show that the second property of Definition 4 holds, namely if $\hat{\text{P}}^{\text{OR}}$ gives an accepting proof with non-negligible probability, then Ext_{OR} fails only with negligible probability. More formally there exists some polynomial $p(\lambda)$ such that for infinitely many λ , $\hat{\text{P}}^{\text{OR}}$ gives an accepting proof with probability $\epsilon(\lambda) \geq \frac{1}{p(\lambda)}$ but the extractor fails to extract the witness with a non-negligible probability.

The extractor Ext_{OR} can fail for two reasons:

1. The extractor of the sigma protocol Ext_Σ fails;
2. Ext_{OR} outputs fail in Step (2).(a).(iii).

The probability of event 1 is negligible since Ext_{OR} perfectly simulates to Ext_Σ a prover with non-negligible success probability $\epsilon(\lambda)$. It suffices to show that the probability of the event 2 is negligible as well. To prove this claim, let us consider a different extractor Ext'_{OR} that behaves like Ext_{OR} with the following difference: Ext'_{OR} executes step (2) once, aborting if $\hat{\text{P}}^{\text{OR}}$ does not send an accepting transcript of Σ^{OR} . Without loss of generality let us assume $\hat{\text{P}}^{\text{OR}}$ to be deterministic. Since $\hat{\text{P}}^{\text{OR}}$ completes with non-negligible probability $\epsilon(\lambda) \geq \frac{1}{q(\lambda)}$ (for some polynomial $q(\lambda)$), we see that, Ext'_{OR} does not abort with probability at least $\epsilon^2(\lambda)$. Since Ext_Σ fails only with negligible probability, then it must be that the extractor Ext'_{OR} fails in Step (2).(a).(iii). This step fails only when $\hat{\text{P}}^{\text{OR}}$ provides two transcripts of Σ^{OR} , namely, $(\text{com}, c = (c_\Sigma, c_{\text{com}}), z = (z^*, \text{op}))$ and $(\text{com}, c' = (c'_\Sigma, c'_{\text{com}}), \hat{z} = (\hat{z}^*, \hat{\text{op}}))$, where the first rounds $(a_i)_{i \in \llbracket n \rrbracket}, (\hat{a}_i)_{i \in \llbracket n \rrbracket}$ of the Σ -protocols obtained from z , and $\hat{z}$ are all different, namely $a_i \neq \hat{a}_i$, for all $i \in \llbracket n \rrbracket$. If this is the case, we can make a reduction to the 1-binding property of $(1, n)$ -SC. The reduction acts as Ext'_{OR} (running in polynomial time $O(q^2(\lambda))$) with $\hat{\text{P}}^{\text{OR}}$ collecting two different openings for $(1, n)$ -SC namely messages $(\text{com}, \hat{\text{op}}, \text{op}, (a_i)_{i \in \llbracket n \rrbracket}, (\hat{a}_i)_{i \in \llbracket n \rrbracket})$ and terminates giving in output those messages.

B On the Communication Complexity of Our Protocol

As examples to evaluate the communication complexity of our protocol in Fig. 7, we consider a disjunction of n instances for two relevant Σ -protocols two NP-complete problems, i.e. (1) Blum's protocol for graph Hamiltonicity [10] and (2) Goldreich, Micali, and Wigderson's protocol for graph 3-coloring [27]. Both Σ -Protocol, where shown to be EHVZK (with a minor modification) in [24] and [12] respectively. We instantiate the prover's commitments with a statistically hiding commitment scheme.

Hamiltonicity: A prover convinces a verifier that they know a Hamiltonian cycle in a given graph.

The prover first constructs a randomly permuted isomorphic copy of the original graph, commits to it, and sends the commitments to the verifier. The verifier then randomly chooses to either verify that the graph is truly an isomorphic copy of the original graph or to check that the prover can reveal a valid Hamiltonian cycle in the committed graph. The prover responds accordingly by opening the commitments either for the full isomorphic mapping or for the Hamiltonian cycle. The protocol is repeated λ times in parallel to make the soundness error negligible. Recall that the communication complexity of our protocol is

$$\begin{aligned} |\text{com}| + |\text{op}| + |c| + |z| &= \\ |\text{com}| + |\text{op}| + |c_{\text{com}}| + |c_\Sigma| + |z| &= \\ l + 12 \cdot l \cdot |r_{\text{SH}}| \cdot \log n + l + |c_\Sigma| + |z| \end{aligned}$$

Where l is the output length of H , and $|r_{\text{SH}}|$ is the size of the opening of SH . Let m be the number of nodes in the graph. The size $|c_\Sigma| = O(\lambda)$, while the size $|z| = O(\lambda) \cdot O(m^2) \cdot O(\lambda) = O(m^2 \lambda^2)$ i.e. all the committed adjacency matrix of size m^2 is opened $O(\lambda)$ times and the size of each opening is $O(\lambda)$. Considering that $l = O(\lambda)$, the final result is

$$O(\lambda) + O(\lambda^2) \cdot \log n + O(\lambda) + O(\lambda) + O(m^2 \lambda^2) = O(\lambda^2 \cdot \log n + m^2 \lambda^2)$$

3-Coloring: A prover convinces a verifier that they know a valid 3-coloring (i.e. no adjacent vertices have the same color) of a given graph with m nodes and t edges without revealing the actual colors. The prover begins by randomly permuting the three colors and committing to the colors of all vertices. The verifier then selects a random edge and challenges the prover to reveal the committed colors of its two endpoints. The prover opens these commitments, and the verifier checks that the two revealed colors are different. Since only one edge is checked per round, the prover can cheat with probability $(1 - 1/t)$. The protocol is repeated $t\lambda$ times in parallel to reduce the soundness cheating probability to negligible. In this case $|z| = O(t\lambda)O(m)O(\lambda) = O(tm\lambda^2)$ i.e., $t\lambda$ commitments to the $O(m)$ nodes of size $O(\lambda)$ are sent. Indeed, to ensure EHVZK (as shown in [12]) we have to modify the protocol and the third round must contain a copy of the first round. As a result, following a similar analysis to the one explained above our final communication complexity is $O(\lambda^2 \cdot \log n + tm\lambda^2)$.

In [12], the communication complexity of their protocol applied on the 3-coloring without parallel repetition (therefore with non-negligible probability of cheating i.e. $1 - 1/t$) is shown to be $O(m\lambda^3 \log(n))$. If we just repeat their protocol $t\lambda$ times in parallel to obtain negligible soundness we get $O(tm\lambda^4 \log(n))$.

C Opening Distribution and Simulator of (1, 2)-bitSC

$\hat{D}_{pp, ccom}^{(op)}$:

1. Sample randomnesses r and parse r as $(\beta, u^b, \text{bit}_0^b, \text{bit}_1^b, b_{\text{swap}}^b, r_s^b, r_k^b)_{b \in \{0,1\}, k \in \llbracket 5 \rrbracket}$ where $\beta, u^b, \text{bit}_0^b, \text{bit}_1^b, b_{\text{swap}}^b \in \{0,1\}$, and all the randomnesses r_k^b are from the randomness space of SH
2. $\forall b \in \{0,1\} \pi^b \leftarrow f_{\text{swap}}(u^b, \langle 0, 1, 2, 3, 4, 5 \rangle)$
3. Compute:
 - (a) $\forall b \in \{0,1\} s^b \leftarrow \text{SH.Commit}(b_{\text{swap}}^b; r_s^b)$
4. $\forall b \in \{0,1\}$ do the following steps:
 - (a) Compute the following:
 - i. $d_0^b \leftarrow \text{SH.Commit}(\text{bit}_0^b; r_{\pi_0^b}^b)$
 - ii. $d_1^b \leftarrow \text{SH.Commit}(\text{bit}_0^b; r_{\pi_1^b}^b)$
 - iii. $d_3^b \leftarrow \text{SH.Commit}(\text{bit}_1^b; r_{\pi_3^b}^b)$
 - iv. $d_4^b \leftarrow \text{SH.Commit}(1 - \text{bit}_1^b; r_{\pi_4^b}^b)$
 - $\backslash \backslash$ For the commitments in $\text{com}^{c_{\text{com}}}$ compute the opening
5. Set $\text{op}_{\text{part}_0} = (u^{c_{\text{com}}} \oplus \beta, r_s^{c_{\text{com}}})$
6. Sample $b \leftarrow \{0,1\}$ and prepare op_β in the following way:
 - If $b = 0$
 - Set $\text{op}_\beta = (\pi_0^{c_{\text{com}}}, \pi_1^{c_{\text{com}}}, (\text{bit}_0^{c_{\text{com}}}, r_{\pi_0^{c_{\text{com}}}}^{c_{\text{com}}}), r_{\pi_2^{c_{\text{com}}}}^{c_{\text{com}}}, d_{\pi_1^{c_{\text{com}}}}^{c_{\text{com}}})$
 - If $b = 1$
 - Set $\text{op}_\beta = (\pi_1^{c_{\text{com}}}, \pi_0^{c_{\text{com}}}, (\text{bit}_0^{c_{\text{com}}}, r_{\pi_1^{c_{\text{com}}}}^{c_{\text{com}}}), r_{\pi_2^{c_{\text{com}}}}^{c_{\text{com}}}, d_{\pi_0^{c_{\text{com}}}}^{c_{\text{com}}})$
7. Sample $b \leftarrow \{0,1\}$ and prepare $\text{op}_{1-\beta}$ in the following way:
 - If $b = 0$:
 - Set $\text{op}_{1-\beta} = (\pi_3^{c_{\text{com}}}, \pi_4^{c_{\text{com}}}, (\text{bit}_1^{c_{\text{com}}}, r_{\pi_3^{c_{\text{com}}}}^{c_{\text{com}}}), r_{\pi_5^{c_{\text{com}}}}^{c_{\text{com}}}, d_{\pi_4^{c_{\text{com}}}}^{c_{\text{com}}}, (b_{\text{swap}}^{c_{\text{com}}}, r_s^{c_{\text{com}}}))$
 - Else:
 - Set $\text{op}_{1-\beta} = (\pi_4^{c_{\text{com}}}, \pi_3^{c_{\text{com}}}, (1 - \text{bit}_1^{c_{\text{com}}}, r_{\pi_4^{c_{\text{com}}}}^{c_{\text{com}}}), r_{\pi_5^{c_{\text{com}}}}^{c_{\text{com}}}, d_{\pi_3^{c_{\text{com}}}}^{c_{\text{com}}}, (b_{\text{swap}}^{c_{\text{com}}}, r_s^{c_{\text{com}}}))$
8. Prepare $\text{op}_{\text{part}_3}$ in the following way
 - $\backslash \backslash$ For the commitments in $\text{com}^{1-c_{\text{com}}}$ open the randomness of the binding position.
 - Set $\text{op}_{\text{part}_3} = (\pi_0^{1-c_{\text{com}}}, \pi_1^{1-c_{\text{com}}}, (\text{bit}_0^{1-c_{\text{com}}}, r_{\pi_0^{1-c_{\text{com}}}}^{1-c_{\text{com}}}), (\text{bit}_0^{1-c_{\text{com}}}, r_{\pi_1^{1-c_{\text{com}}}}^{1-c_{\text{com}}}), d_{\pi_k^{1-c_{\text{com}}}}^{1-c_{\text{com}}} \}_{k \in \{2,3,4,5\}}, s^{1-c_{\text{com}}})$
9. If $u_{\text{com}}^c = 0$ set $\text{op}_{\text{part}_1} = \text{op}_\beta, \text{op}_{\text{part}_2} = \text{op}_{1-\beta}$, else $\text{op}_{\text{part}_1} = \text{op}_{1-\beta}, \text{op}_{\text{part}_2} = \text{op}_\beta$
 - $\backslash \backslash$ Prepare the openings in the correct order.
10. Output $\text{op} = (\text{op}_{\text{part}_0}, \text{op}_{\text{part}_1}, \text{op}_{\text{part}_2}, \text{op}_{\text{part}_3})$

Fig. 8: Definition of the distribution $\hat{D}_{pp, ccom}^{(op)}$ of our (1,2)-bitSC.

$\text{WSim}_{\text{com}}(c_{\text{com}}, m_0, m_1, \text{op})$

1. Parse op as $(\text{op}_{\text{part}_0}, \text{op}_{\text{part}_1}, \text{op}_{\text{part}_2}, \text{op}_{\text{part}_3})$ with:
 - $\text{op}_{\text{part}_0}$: $b_{\text{swap}}, r_s^{c_{\text{com}}}$
 - $\text{op}_{\text{part}_1}$: $i_0, i_1, \text{bit}_0^{c_{\text{com}}}, r_{i_0}^{c_{\text{com}}}, r_2^{c_{\text{com}}}, e_{i_1}^{c_{\text{com}}}$
 - $\text{op}_{\text{part}_2}$: $k_0, k_1, \text{bit}_1^{c_{\text{com}}}, r_{k_0}^{c_{\text{com}}}, r_5^{c_{\text{com}}}, e_{k_1}^{c_{\text{com}}}$
 - $\text{op}_{\text{part}_3}$: $l_0, l_1, (\text{bit}_0^{1-c_{\text{com}}}, r_{l_0}^{1-c_{\text{com}}}), (\text{bit}_0^{1-c_{\text{com}}}, r_{l_1}^{1-c_{\text{com}}}), \{\text{e}_k^{1-c_{\text{com}}}\}_{k \in \{2,3,4,5\}}, \widetilde{s^{1-c_{\text{com}}}}$
2. Compute the following steps:
 - $\backslash \backslash$ Compute $(\text{com}^{c_{\text{com}}}, s^{c_{\text{com}}})$ starting from the openings.
 - (a) $s^{c_{\text{com}}} \leftarrow \text{SH.Commit}(b_{\text{swap}}; r_s^{c_{\text{com}}})$
 - (b) $d_{i_0}^{c_{\text{com}}} \leftarrow \text{SH.Commit}(\text{bit}_0^{c_{\text{com}}}; r_{i_0}^{c_{\text{com}}})$
 - (c) $d_2^{c_{\text{com}}} \leftarrow \text{SH.Commit}(v_0^{c_{\text{com}}}; r_2^{c_{\text{com}}})$, where $v_0^{c_{\text{com}}} = m_{b_{\text{swap}}} \oplus \text{bit}_0^{c_{\text{com}}}$
 - (d) $d_{k_0}^{c_{\text{com}}} \leftarrow \text{SH.Commit}(\text{bit}_1^{c_{\text{com}}}; r_{k_0}^{c_{\text{com}}})$
 - (e) $d_5^{c_{\text{com}}} \leftarrow \text{SH.Commit}(v_1^{c_{\text{com}}}; r_5^{c_{\text{com}}})$, where $v_1^{c_{\text{com}}} = m_{1-b_{\text{swap}}} \oplus \text{bit}_1^{c_{\text{com}}}$
 - (f) Set $d_{i_1}^{c_{\text{com}}} = e_{i_1}^{c_{\text{com}}}$
 - (g) Set $d_{k_1}^{c_{\text{com}}} = e_{k_1}^{c_{\text{com}}}$
3. Set $\text{com}^{c_{\text{com}}} = (d_0^{c_{\text{com}}}, d_1^{c_{\text{com}}}, d_2^{c_{\text{com}}}, d_3^{c_{\text{com}}}, d_4^{c_{\text{com}}}, d_5^{c_{\text{com}}})$
4. Compute the following steps:
 - $\backslash \backslash$ Compute $(\text{com}^{1-c_{\text{com}}}, s^{1-c_{\text{com}}})$ starting from the opening for the 1-challenge.
 - (a) $d_{l_0}^{1-c_{\text{com}}} \leftarrow \text{SH.Commit}(\text{bit}_0^{1-c_{\text{com}}}; r_{l_0}^{1-c_{\text{com}}})$
 - (b) $d_{l_1}^{1-c_{\text{com}}} \leftarrow \text{SH.Commit}(\text{bit}_0^{1-c_{\text{com}}}; r_{l_1}^{1-c_{\text{com}}})$
 - (c) Set $d_2^{1-c_{\text{com}}} = e_2^{1-c_{\text{com}}}$
 - (d) Set $d_3^{1-c_{\text{com}}} = e_3^{1-c_{\text{com}}}$
 - (e) Set $d_4^{1-c_{\text{com}}} = e_4^{1-c_{\text{com}}}$
 - (f) Set $d_5^{1-c_{\text{com}}} = e_5^{1-c_{\text{com}}}$
 - (g) Set $s^{1-c_{\text{com}}} = \widetilde{s^{1-c_{\text{com}}}}$
5. Set $\text{com}^{1-c_{\text{com}}} = (d_0^{1-c_{\text{com}}}, d_1^{1-c_{\text{com}}}, d_2^{1-c_{\text{com}}}, d_3^{1-c_{\text{com}}}, d_4^{1-c_{\text{com}}}, d_5^{1-c_{\text{com}}})$
6. Output $(\text{com}^{c_{\text{com}}}, \text{com}^{1-c_{\text{com}}}, s^{c_{\text{com}}}, s^{1-c_{\text{com}}})$

Fig. 9: Simulator of our (1, 2)-bitSC.